# Organização e Arquitetura de Computadores - OAC

# Parte 3

***"A experiência é resultado da prática".***

***"A vontade de vencer é importante, mas a vontade de se preparar é vital".***

Prof. Misael Morais

Moraiscg.uepb@gmail.com

**http://sites.google.com/site/moraiscg**

# Álgebra Boolena

- Conceito

- Álgebra Booleana
  - **Postulados (convenções)**
    - Elementos: “0” e “1”
    - Soma (+)
    - Produto (.)
    - Negação ($\overline{\ }$)
  - **Leis (axiomas)**
    - Comutativa: X.Y = Y.X
    - Associativa: X.(Y + Z) = X.Y + X.Z
    - Distributiva: X + (Y + Z) = (X + Y) + Z

- (Cont.) Álgebra Booleana
  - Dualidade: $(1 + X).(Y + 0)) = Y$ e $0.X + Y.1 = Y$
- **Teoremas booleanos**
  - Adição: $X + 0 = X$; $X+1 = 1$
  - Multiplicação: $X.0 = 0$; $X.1=X$
  - De Morgan: $\overline{X.Y} = \overline{X} + \overline{Y}$; $\overline{X + Y} = \overline{X}.\overline{Y}$
  - ...
- Tabela Verdade (Formas Canônicas)
- Somas de Produtos: **MINITERMO** - são termos somente com AND (termos **PRODUTO**)
- Produtos de Somas: **MAXITERMO** - são termos somente com OR (termos **SOMA**)

# Representação: termos PRODUTO e SOMA

Minitermos

- Exemplo: SOMA DE PRODUTO

$$\sum(0, 3, 5, 6, 7)$$

Maxitermos

- Exemplo: PRODUTO DE SOMA

$$\prod(1, 5, 6, 9, 11, 15)$$

# Tabela Verdade

- $n$ entradas $\Rightarrow 2^n$ possíveis valores de saída
- tabela verdade correspondente $2^n$ linhas
- funções lógicas mais simples

| A | f (NOT) |
|---|---|
| 0 | 1 |
| 1 | 0 |

| A | B | f (AND) |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

| A | B | f (OR) |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

- descrevem completamente qualquer função lógica combinacional
- inviável quando número de variáveis é muito grande

# Exemplo:

| A | B | C | f | MINITERMOS | MAXITERMOS |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 1 | $\bar{A}.\bar{B}.\bar{C}$ + | |
| 0 | 0 | 1 | 0 | | $A+B+\bar{C}$ . |
| 0 | 1 | 0 | 0 | | $A+\bar{B}+C$ . |
| 0 | 1 | 1 | 1 | $\bar{A}.B.C$ + | |
| 1 | 0 | 0 | 0 | | $\bar{A}+B+C$ . |
| 1 | 0 | 1 | 1 | $A.\bar{B}.C$ + | |
| 1 | 1 | 0 | 0 | | $\bar{A}+\bar{B}+C$ . |
| 1 | 1 | 1 | 1 | $A.B.C$ | |

Soma dos minitermos

$$f = \bar{A}.\bar{B}.\bar{C} + \bar{A}.B.C + A.\bar{B}.C + A.B.C$$

Produto dos maxitermos

$$f = (A+B+\bar{C}).(A+\bar{B}+C).(\bar{A}+B+C).(\bar{A}+\bar{B}+C)$$

# Outra forma de representar

## Considerando soma de produtos

1. Cobrir os "1s"
2. Utilizar as posições com *don't care* para encontrar a melhor cobertura
3. Cada posição com *don't care* é totalmente independente das demais

**Exemplo**

Mapa de Karnaugh

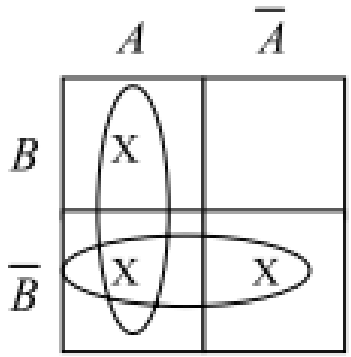
| F29 | $\bar{C}\bar{D}$ | $\bar{C}D$ | $CD$ | $C\bar{D}$ |
|---|---|---|---|---|
| $\bar{A}\bar{B}$ | 1 | 1 | X | 1 |
| $\bar{A}B$ | 0 | 0 | X | 0 |
| $AB$ | 1 | 1 | X | X |
| $A\bar{B}$ | 0 | 0 | 0 | 0 |

- Funções lógicas que contém valores
  - que não interessam
  - nunca ocorrem
- *Don´t cares* ⇒ facilitam o processo de otimização da implemetação de funções lógicas

# MINIMIZAÇÃO DE CIRCUITOS LÓGICOS

- Expressões Booleanas
- Mapas de Karnaugh

$$E = AB + A\overline{B} + \overline{B}$$



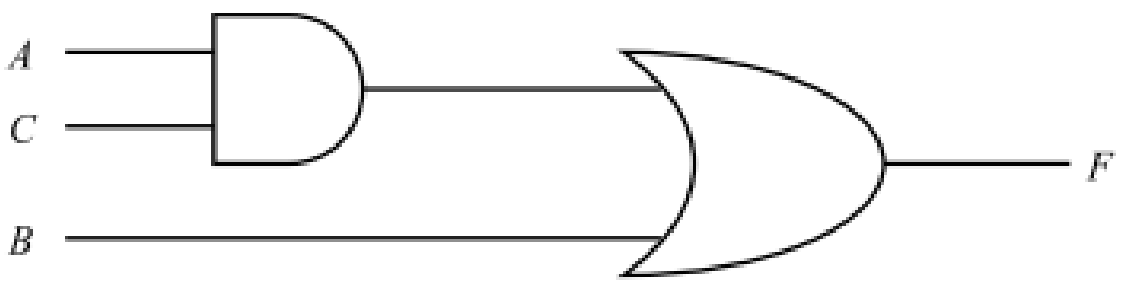

$$F = A + \overline{B}$$

# Exemplo: Mapa de Karnaugh

$$E = ABC + A\overline{B}C + AB + \overline{A}B$$

| | $AB$ | $A\overline{B}$ | $\overline{A}\,\overline{B}$ | $\overline{A}B$ |
|---|---|---|---|---|
| $C$ | X | X | | X |
| $\overline{C}$ | X | | | X |

$$F = AC + B$$



# Funções com *Don´t cares*

Como fazer a cobertura em Soma de Produtos

- O objetivo é cobrir os "1s" da função
- Posições com *don't care* podem ser usadas para ajudar a melhorar a cobertura dos "1s"
- Cada posição com *don't care* é totalmente independente das demais

# Funções com *Don´t cares*

**Como fazer a cobertura em Produto de Somas**

- **O objetivo é cobrir os "0s" da função**
- **Posições com *don't care* podem ser usadas para ajudar a melhorar a cobertura dos "0s"**
- **Cada posição com *don't care* é totalmente independente das demais**

# Funções com *Don´t cares*

## Considerando produto de somas

1. Cobrir os "**0s**"
2. Utilizar as posições com *don't care* para encontrar a melhor cobertura
3. Cada posição com *don't care* é totalmente independente das demais

**Exemplo**

| F29 | $\bar{C}\bar{D}$ | $\bar{C}D$ | $CD$ | $C\bar{D}$ |
|---|---|---|---|---|
| $\bar{A}\bar{B}$ | 1 | 1 | X | 1 |
| $\bar{A}B$ | 0 | 0 | X | 0 |
| $AB$ | 1 | 1 | X | X |
| $A\bar{B}$ | 0 | 0 | 0 | 0 |

# Funções com *Don´t cares*

## Exercício

- Encontre a equação mínima em produto de somas para a função abaixo
- Desenhe o circuito resultante e calcule seu custo

$$S13(A,B,C,D) = \sum(0, 3, 5, 6, 7) + DC(10, 11, 12, 13, 14, 15)$$

| S13 | $\bar{C}\bar{D}$ | $\bar{C}D$ | $CD$ | $C\bar{D}$ |
|---|---|---|---|---|
| $\bar{A}\bar{B}$ | 1 | 0 | 1 | 0 |
| $\bar{A}B$ | 0 | 1 | 1 | 1 |
| $AB$ | X | X | X | X |
| $A\bar{B}$ | 0 | 0 | X | X |

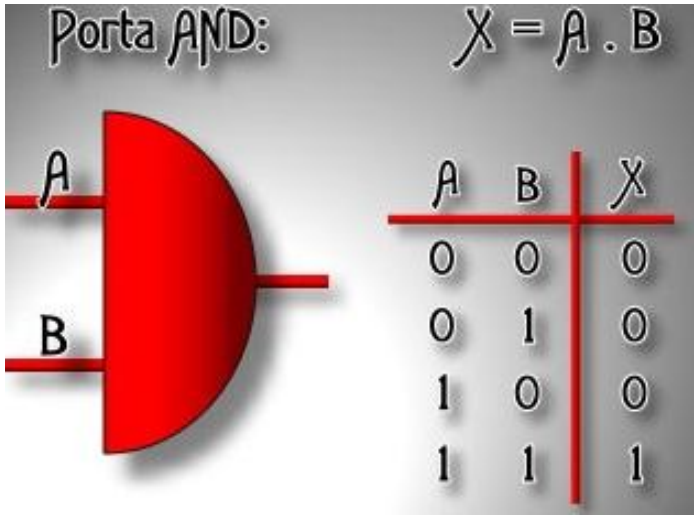
# Relógios

- *Clocks* ⇒ sinal periódico, com tempo de ciclo fixo
- Freqüencia = inverso do período
- Período dividido em: nível alto e baixo
- Circuitos sensíveis:
  - transições: positivas ou negativas
  - nível: alto baixo

# Implementando circuitos a partir de expressões booleanas

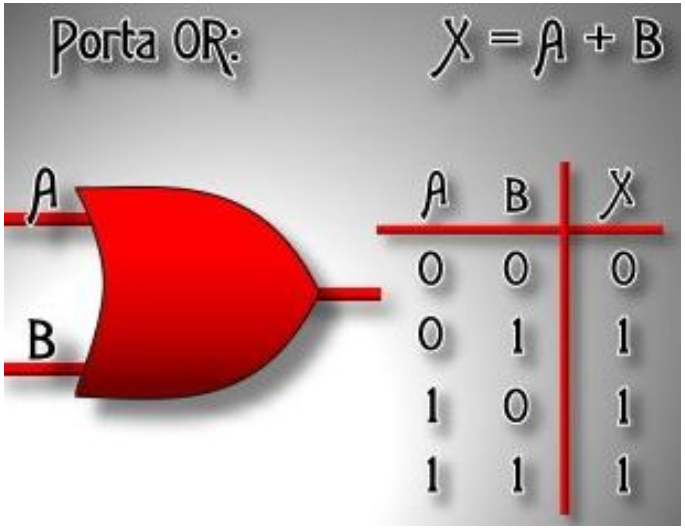
- Operações Lógicas (portas lógicas)

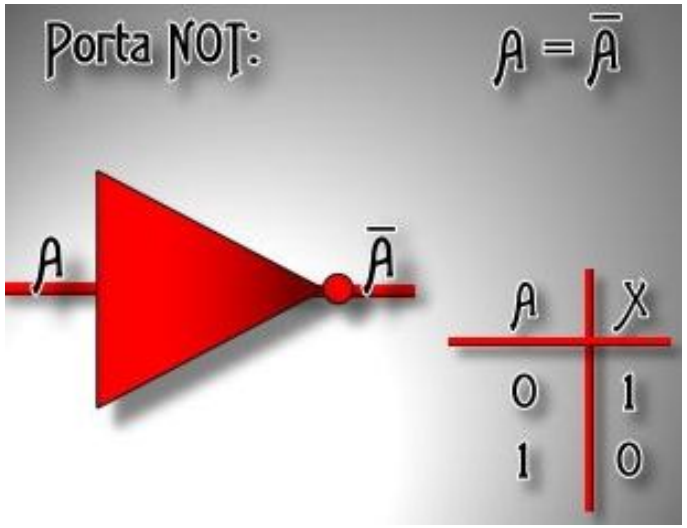
**AND (E):**



# Operações Lógicas (portas lógicas)

- **OR (OU):**

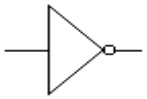



- **<u>NOT (NÃO):</u>**

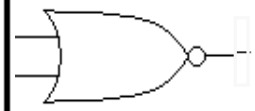
# Operações Lógicas (portas lógicas)

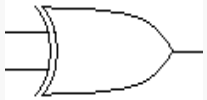
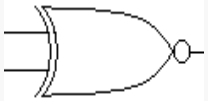
- Simbologia

| BLOCOS LÓGICOS BÁSICOS | | | | |
|---|---|---|---|---|
| **Porta** | **Símbolo usual** | **Tabela Verdade** | **Função Lógica** | **Expressão** |
| E<br>AND | | A B S<br>0 0 0<br>0 1 0<br>1 0 0<br>1 1 1 | **Função E**: assume 1 quando todas as variáveis forem 1 e 0 nos outros casos. | $S = A \cdot B$ |
| OU<br>OR | | A B S<br>0 0 0<br>0 1 1<br>1 0 1<br>1 1 1 | **Função OU**: assume 0 quando todas as variáveis forem 0 e 1 nos outros casos. | $S = A + B$ |
| NÃO<br>NOT |  | A S<br>0 1<br>1 0 | **Função NOT**: inverte a variável aplicada a sua entrada. | $S = \overline{A}$ |

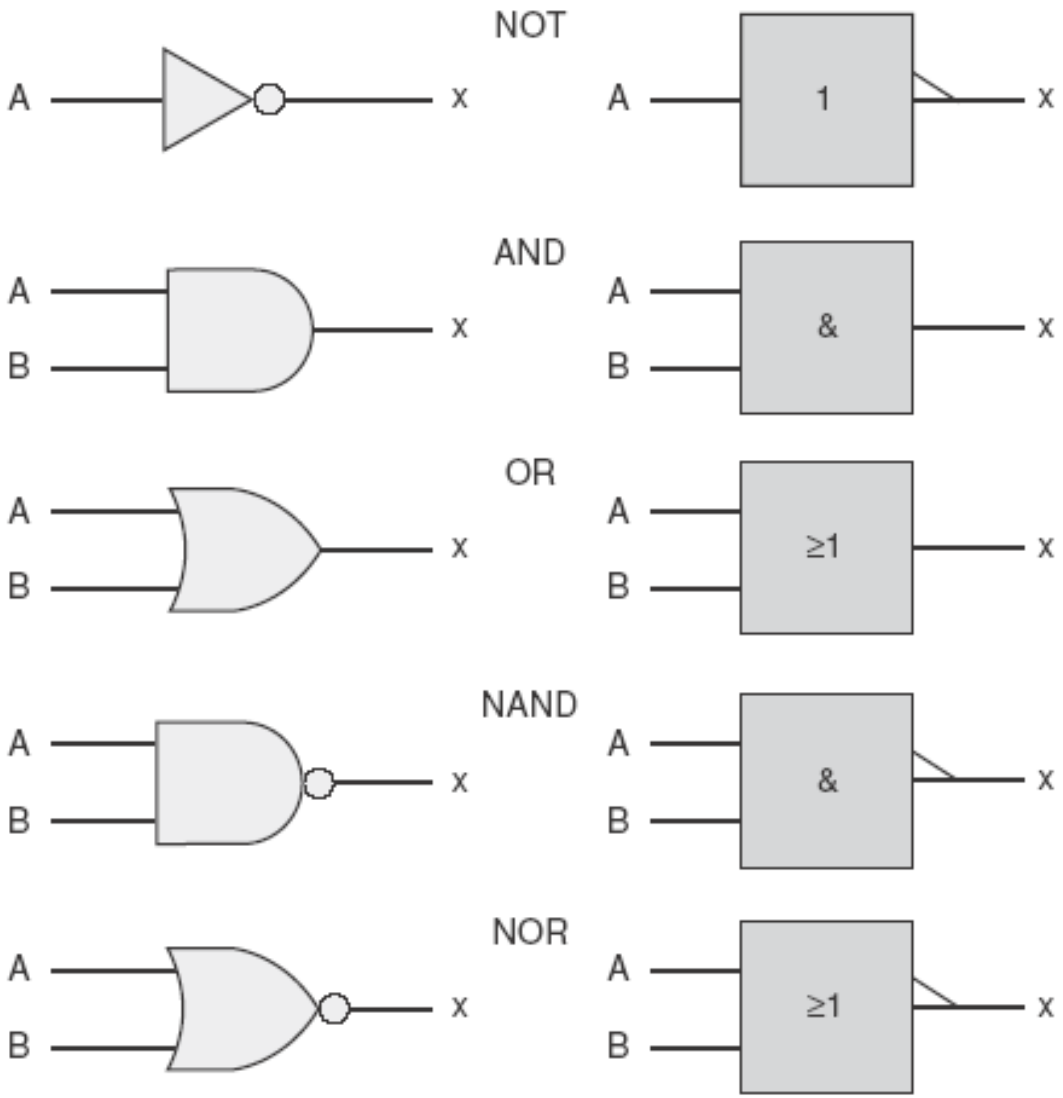
# Operações Lógicas (portas lógicas)

- Simbologia

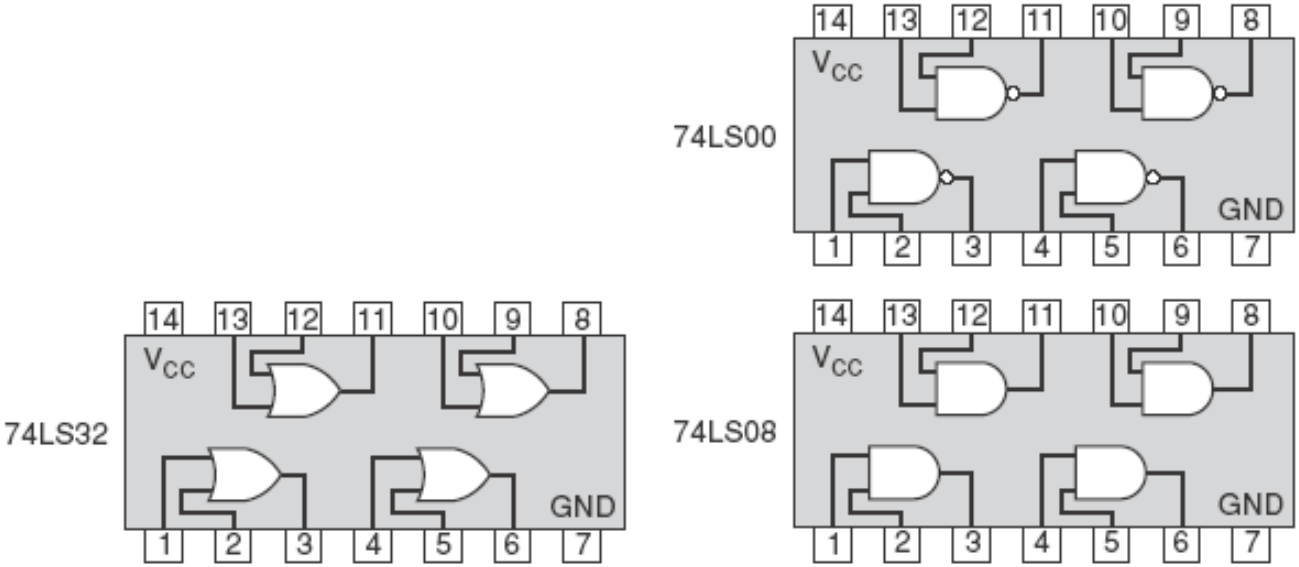
| BLOCOS LÓGICOS BÁSICOS | | | | |
|---|---|---|---|---|
| **Porta** | **Símbolo usual** | **Tabela Verdade** | **Função Lógica** | **Expressão** |
| NÃO E<br>NAND | | A B S<br>0 0 1<br>0 1 1<br>1 0 1<br>1 1 0 | **Função NÃO E**: Inverso da função E | $S = \overline{A \cdot B}$ |
| NÃO OU<br>NOR |  | A B S<br>0 0 1<br>0 1 0<br>1 0 0<br>1 1 0 | **Função NÃO OU**: Inverso da função OU | $S = \overline{A + B}$ |

| BLOCOS LÓGICOS BÁSICOS | | | | |
|---|---|---|---|---|
| **Porta** | **Símbolo usual** | **Tabela Verdade** | **Função Lógica** | **Expressão** |
| OU EXCLUSIVO<br>XOR |  | A B S<br>0 0 0<br>0 1 1<br>1 0 1<br>1 1 0 | **Função XOR**: Assume 1 quando as duas variáveis assumirem valores diferentes entre si. | $S = A \oplus B$ |
| COINCIDÊNCIA<br>XNOR |  | A B S<br>0 0 1<br>0 1 0<br>1 0 0<br>1 1 1 | **Função XNOR**: Assume 1 quando houver coincidência entre os valores das variáveis. | $S = A \otimes B$ |

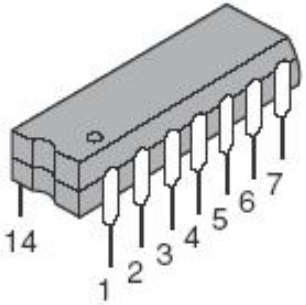
# Símbolos padrão IEEE/ANSI (1984)

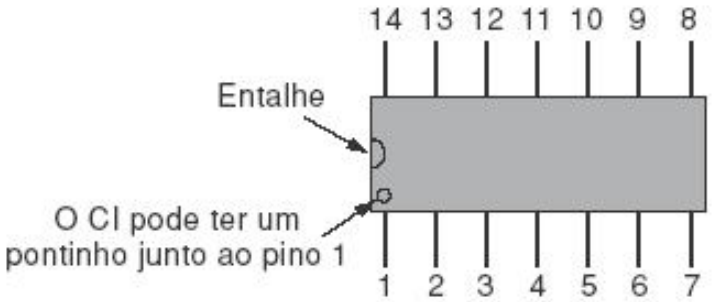


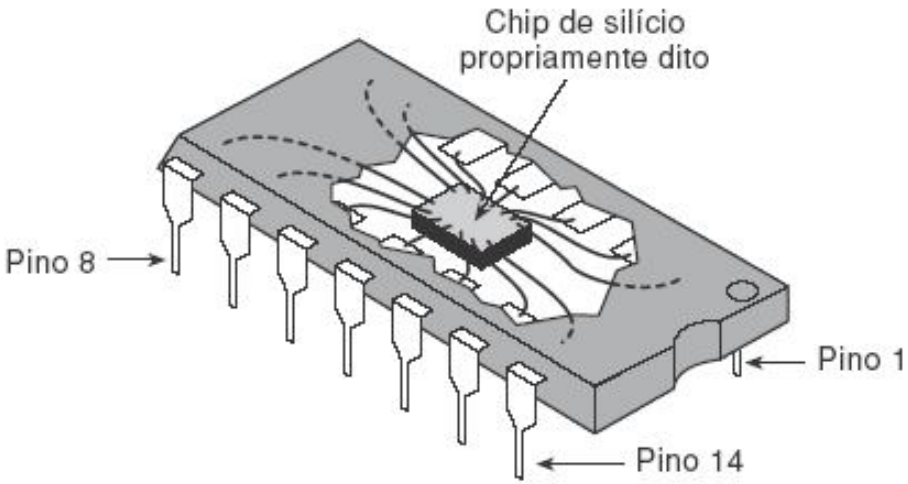
# Encapsulamento dual-in-line (DIP)

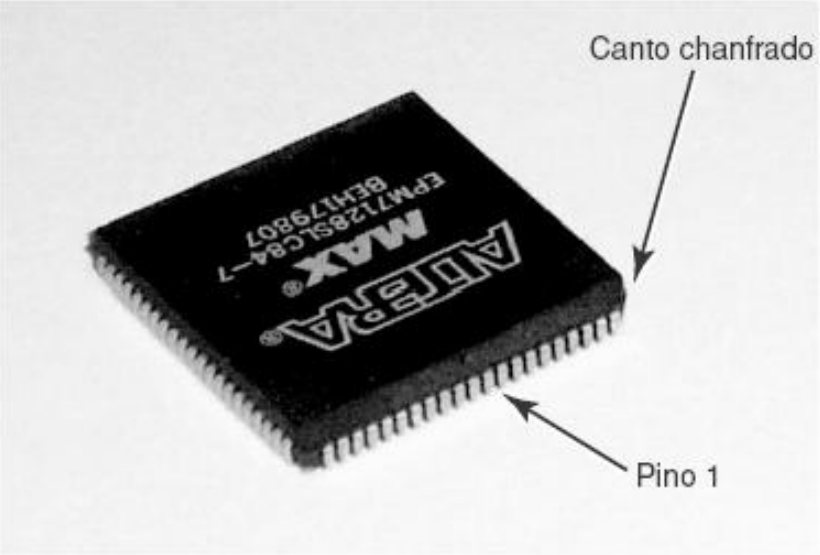


# Encapsulamento dual-in-line (DIP) e PLCC



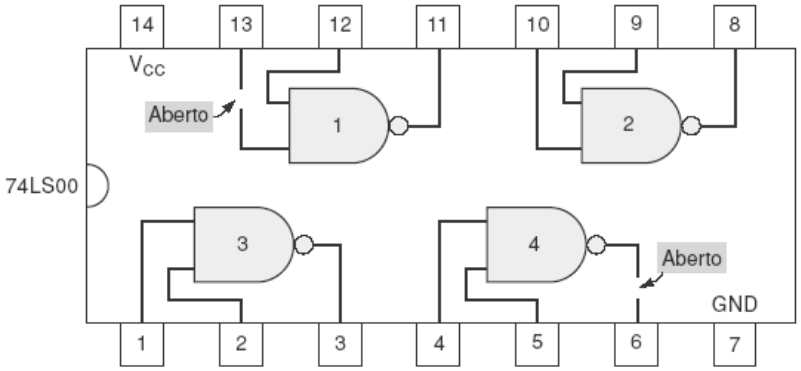
# Falhas dos Cis digitais

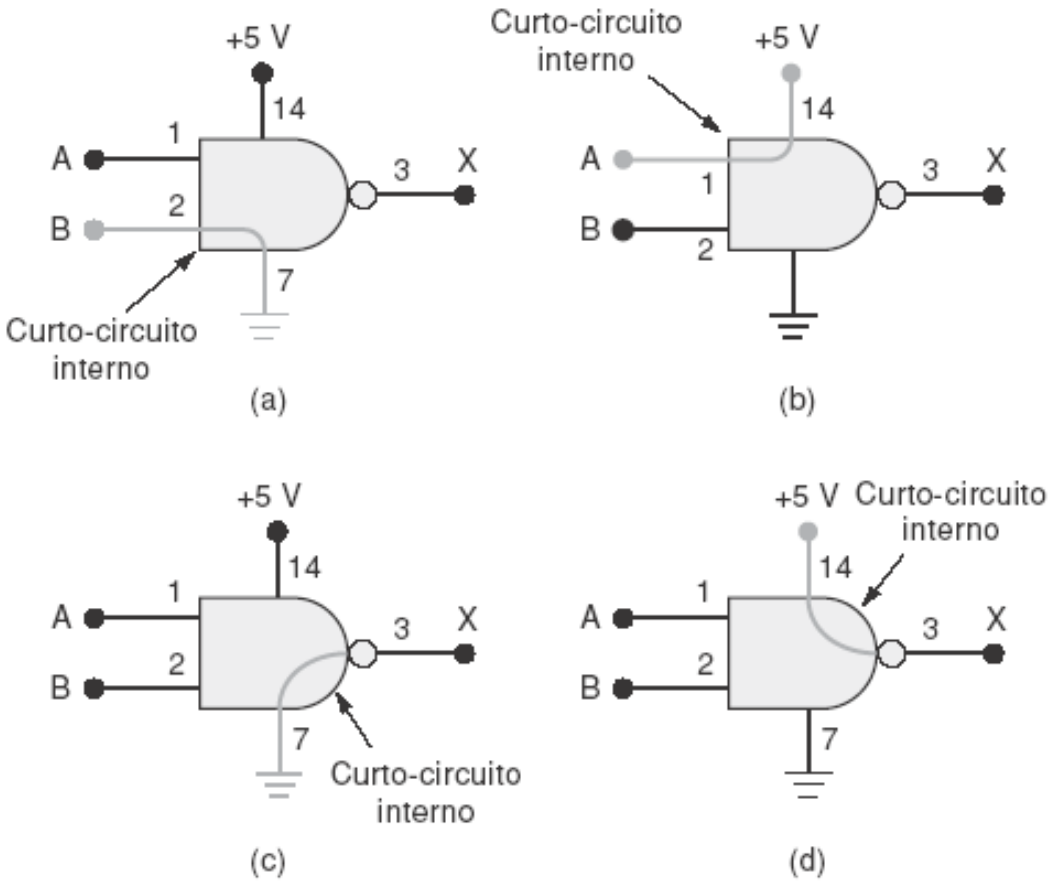
a) Mau funcionamento do circuito interno do CI

b) Entradas ou saídas curto-circuitadas para GND ou $V_{cc}$

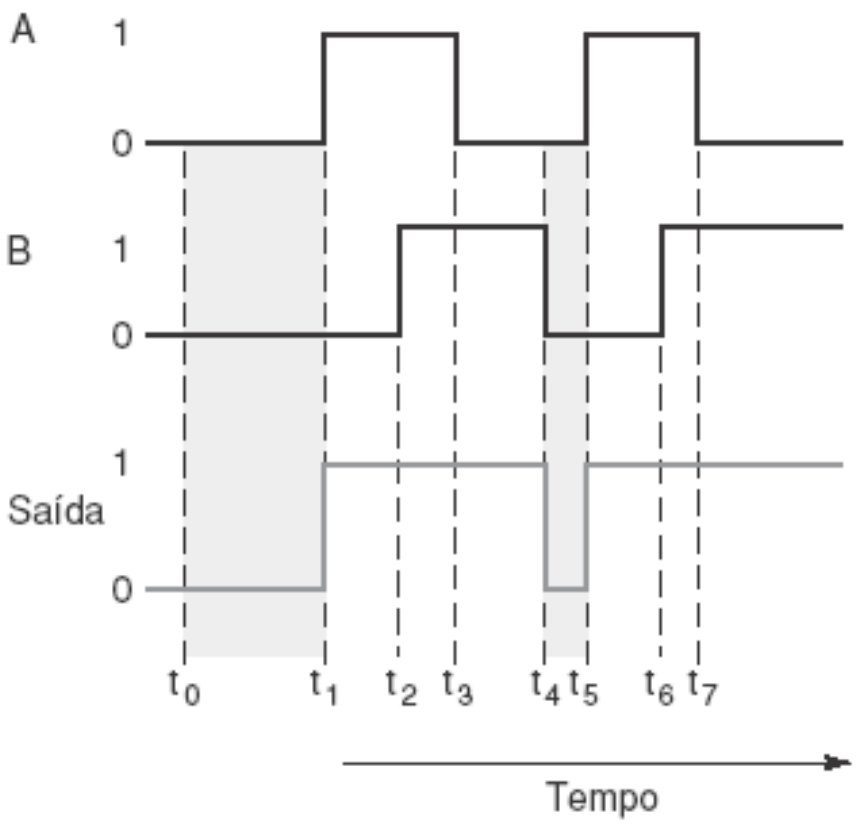
c) Entradas ou saídas abertas

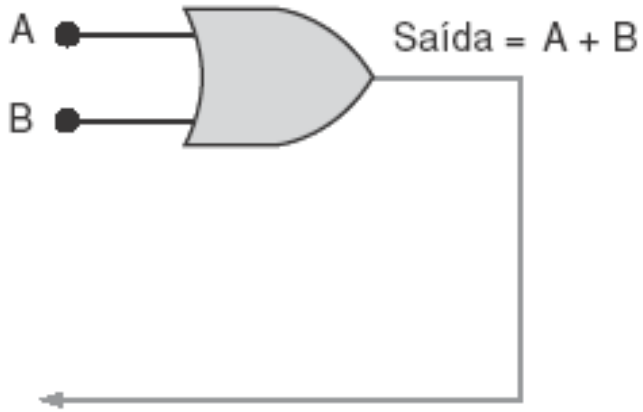
d) Curto-circuito entre dois pinos (exceto GND ou $V_{cc}$)



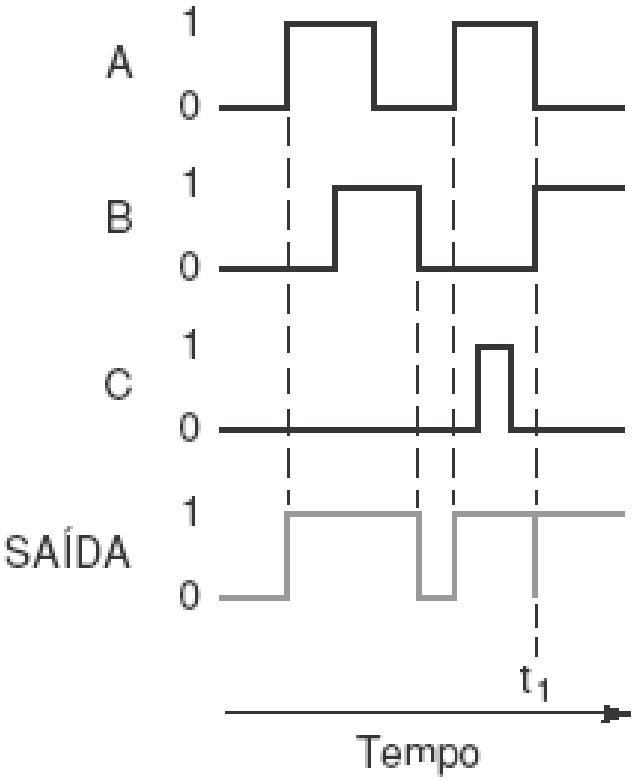



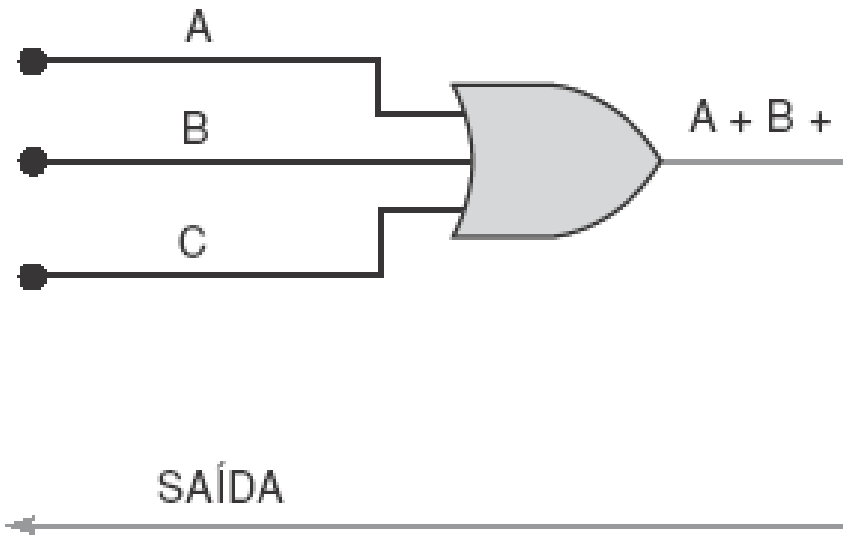
# Exemplos



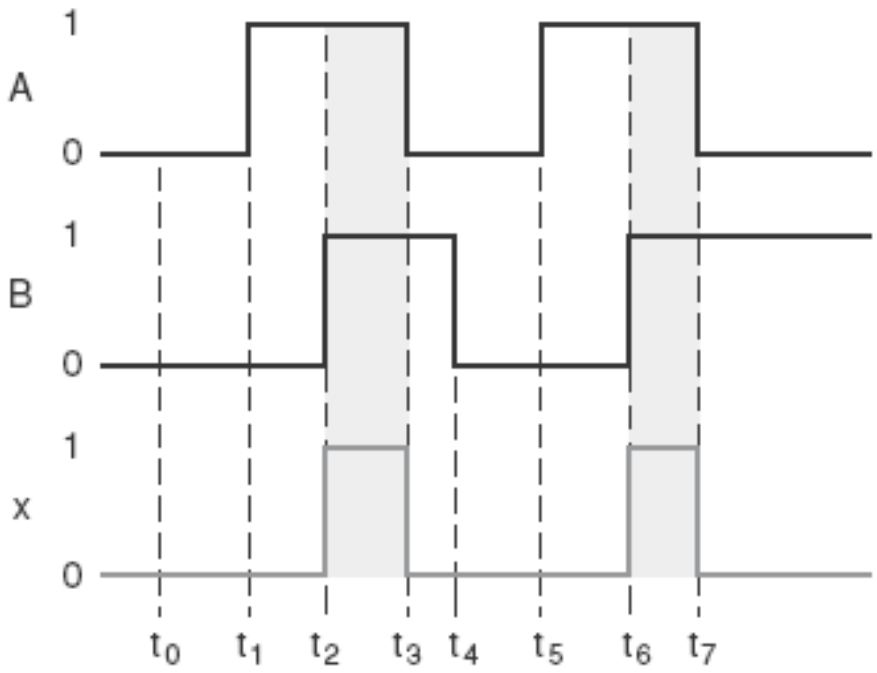



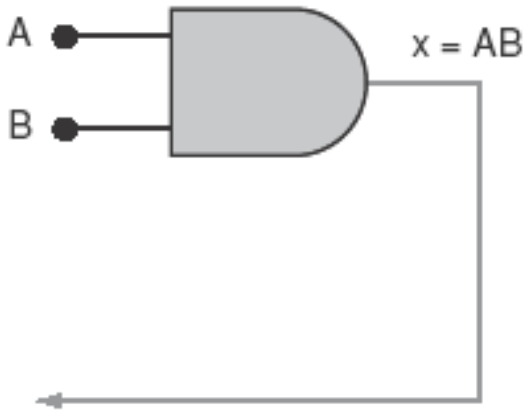
# Exemplo

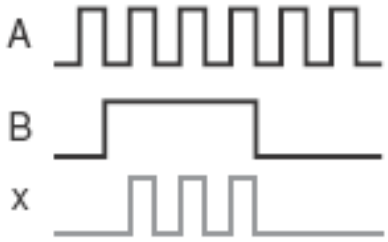


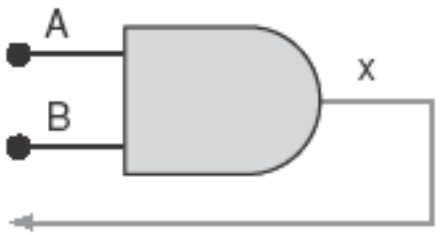



# Exemplo



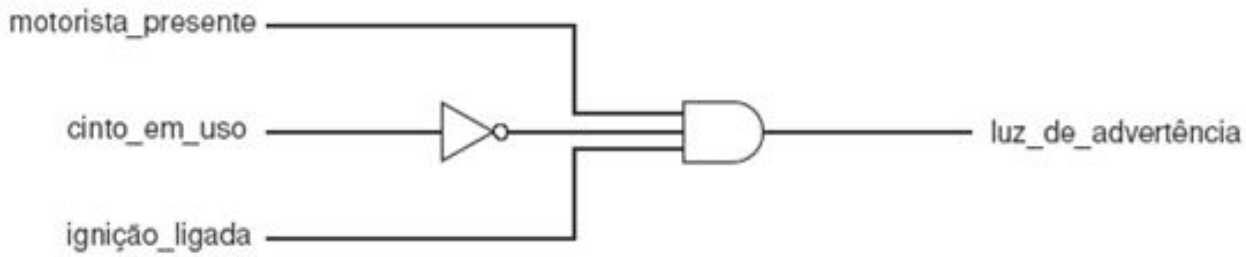



**Expressão booleana**

luz_de_advertência = motorista_presente • $\overline{\text{cinto_em_uso}}$ • ignição_ligada

(a)

**Diagrama esquemático**



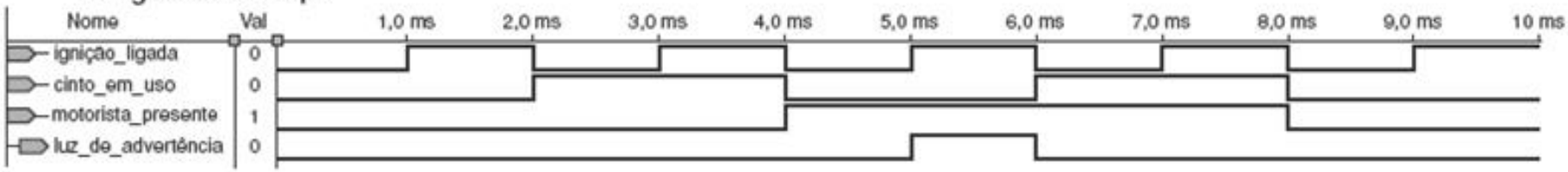

(b)

**Tabela-verdade**

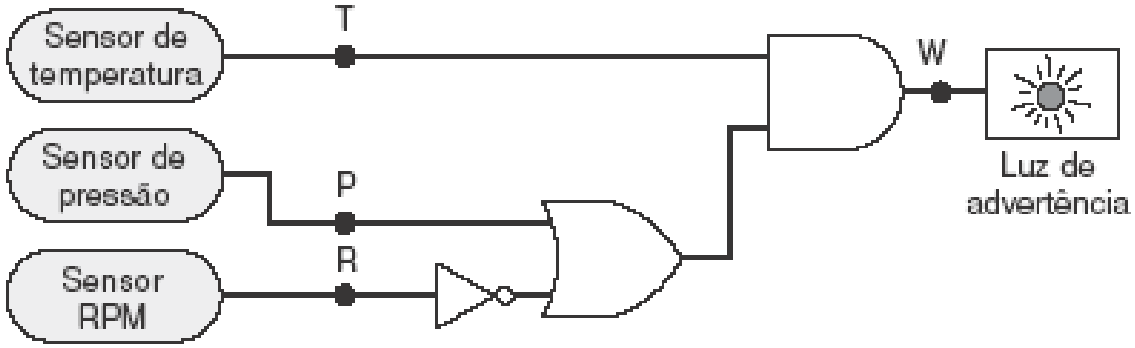
| motorista_presente | cinto_em_uso | ignição_ligada | luz_de_advertência |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 0 |
| 1 | 1 | 1 | 0 |

(c)

**Diagrama de tempo**




(d)


Sensor de temperatura
T
Sensor de pressão
P
Sensor RPM
R
W
Luz de advertência

# Circuitos lógicos digitais

Podem ou não conter memória

- Combinacionais
  - **saída** ⇒ entrada corrente
  - representação: tabela verdade
- Seqüenciais
  - **saída** ⇒ entrada corrente e estado anterior armazenado (estado)
  - representação: máquina de estados finitos

# Elementos de Memória

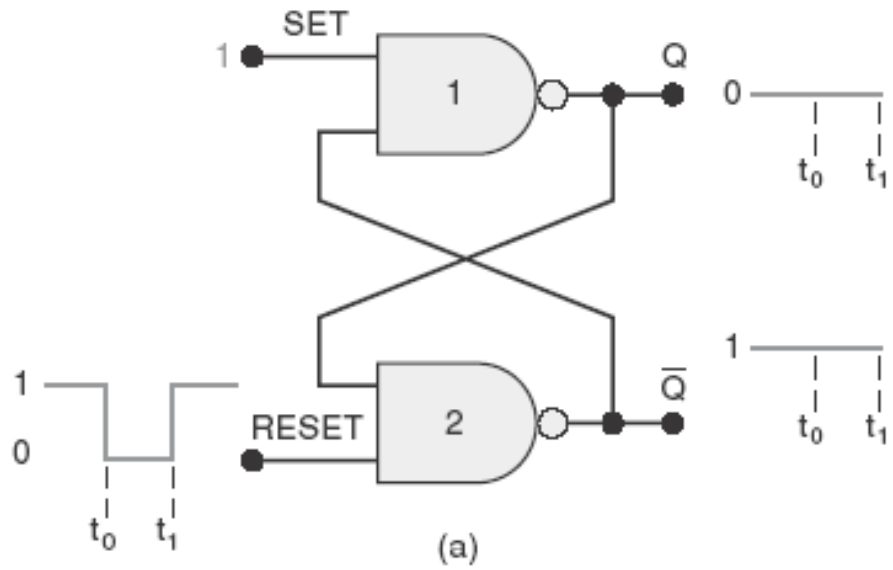
- Armazenam estado ⇒ a sua saída depende tanto das entradas quanto do valor armazenado anteriormente nesse elemento
- Circuitos seqüenciais ⇒ exemplos: *latches*, *flip-flops*, registradores e memórias
- *Flip-flops*: tipo D, JK, *toggle*
- Conjunto de *flip-flops* do tipo D ⇒ construir registradores para armazenar um dado com vários *bits.*

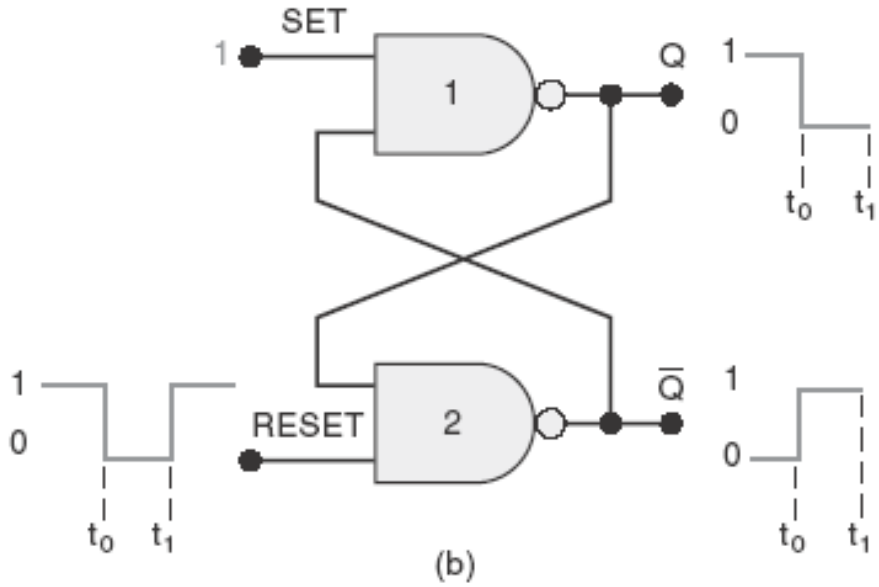
# Máquinas de Estados Finitos

- Sistemas Seqüenciais contêm estados armazenados em elementos de memória internos ⇒ não pode ser expresso através de tabela verdade

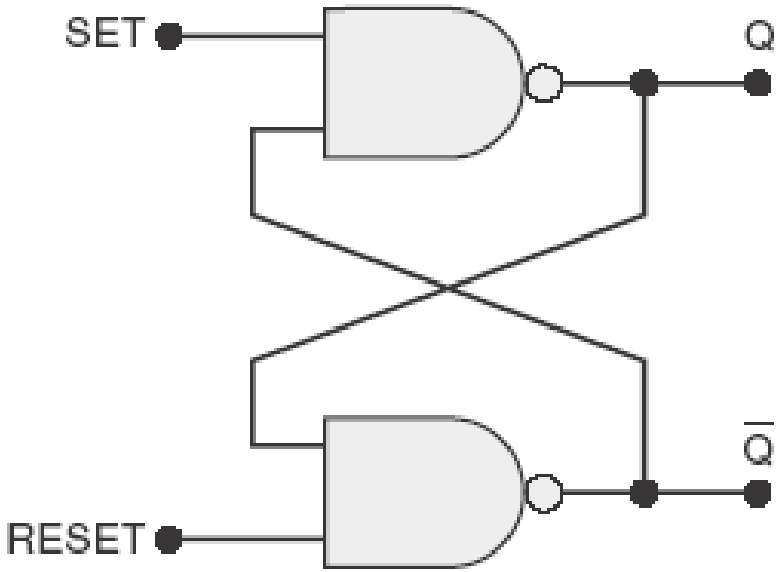
- máquinas de estados finitos
  - Conjunto de Estados ⇒ todos os possíveis valores que a memória interna pode assumir n *bits* ⇒ $2^n$ estados
  - duas funções:
    - próximo estado: função combinacional das entradas e do estado corrente
    - saída: função saída produz um conjunto de saídas a partir do estado atual e das entradas

# Circuito sequencial



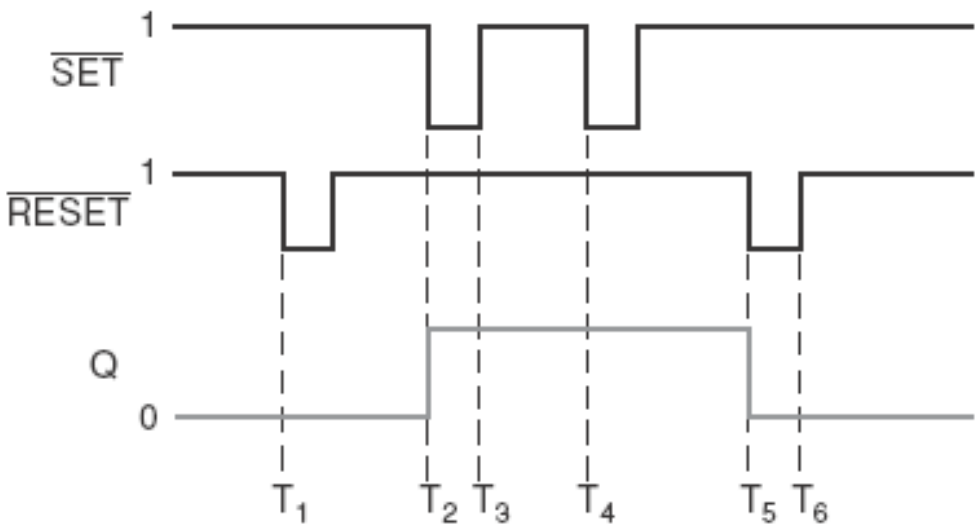



# Latch




| Set | Reset | Saída |
|---|---|---|
| 1 | 1 | Não muda |
| 0 | 1 | Q = 1 |
| 1 | 0 | Q = 0 |
| 0 | 0 | Inválida* |

*Produz $Q = \overline{Q} = 1$.



# Dispositivos Lógicos Programáveis – PLD

**São circuitos lógicos digitais que podem ter suas funções booleanas programadas**

# Evolução dos DLP

- FPD - (DLP) Dispositivo programável por campo
- PLA- Array lógico programável
- CPLD- um PLD mais complexo que consiste em um arranjo de múltiplos blocos
- FPGA- Field-Programmable Gate Array

# Sistema de desenvolvimento com PLD

# Organização e Arquitetura de Computadores

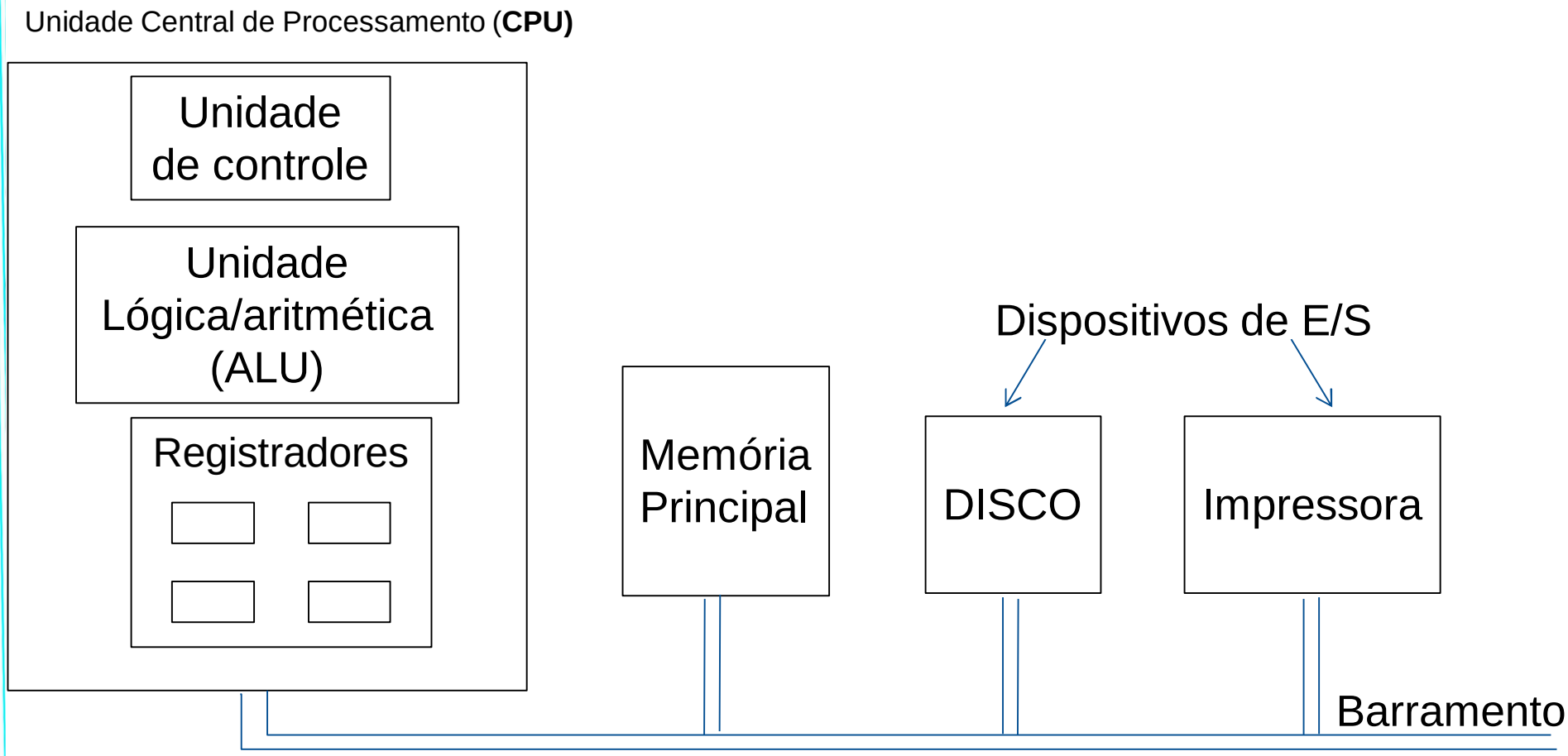
Cap.3 –Stalling)

- **Organização de Computadores** está focada na forma com que os componentes de hardware estão interconectados para formar um sistema computacional.

- **Arquitetura de Computadores** preocupa-se com a estrutura e comportamento dos vários módulos funcionais de um computador, e como eles interagem para atender às necessidades de processamento do usuário.

# Organização Básica

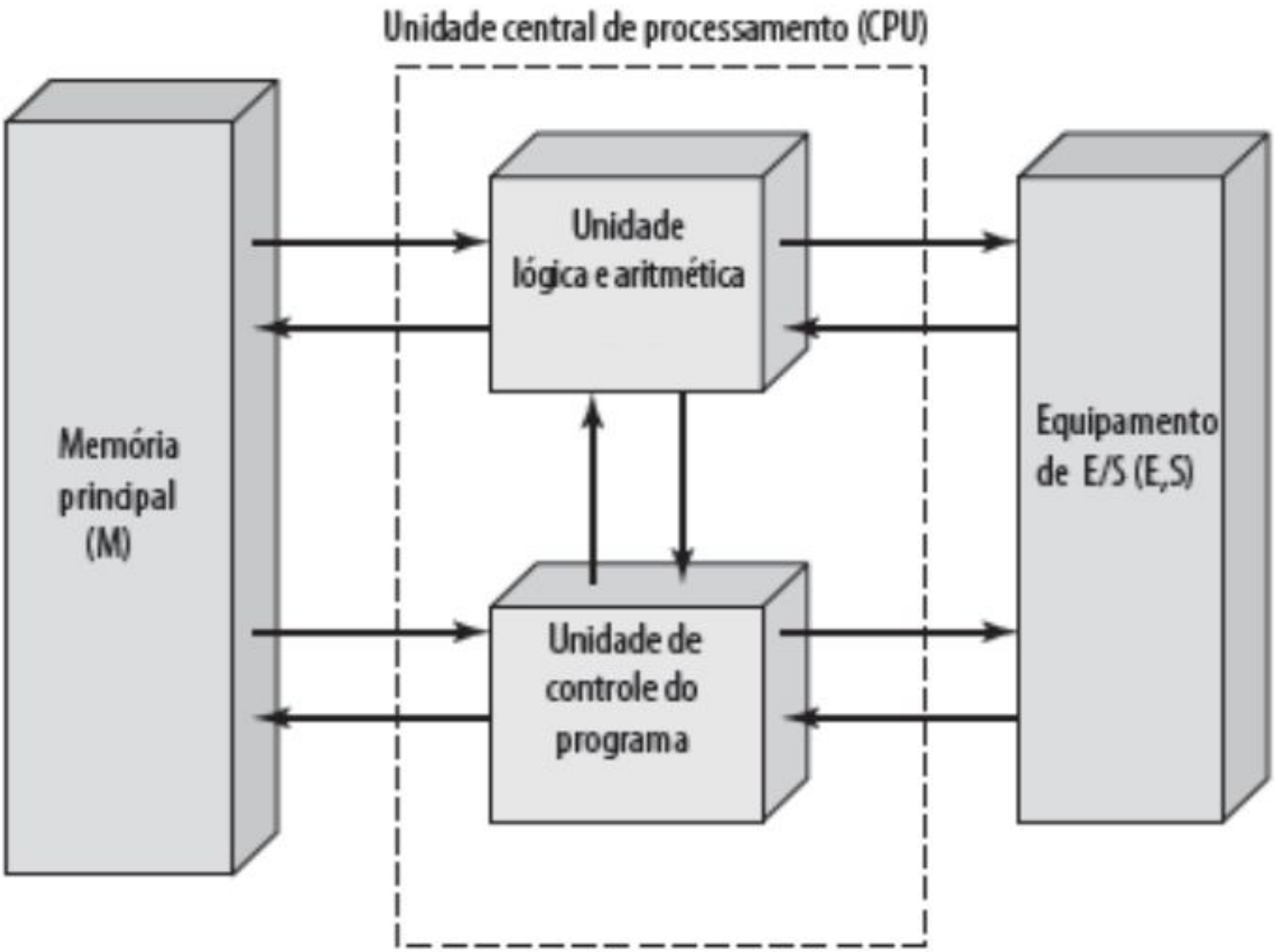
- Composição básica de um Computador digital (John Von Neumann)



# Conceito da arquitetura von Neumann

- Dados e instruções armazenados na mesma memória
- O conteúdo da memória é endereçada pela sua posição
- A execução da instrução ocorre de modo sequencial

# Conceito da arquitetura von Neumann

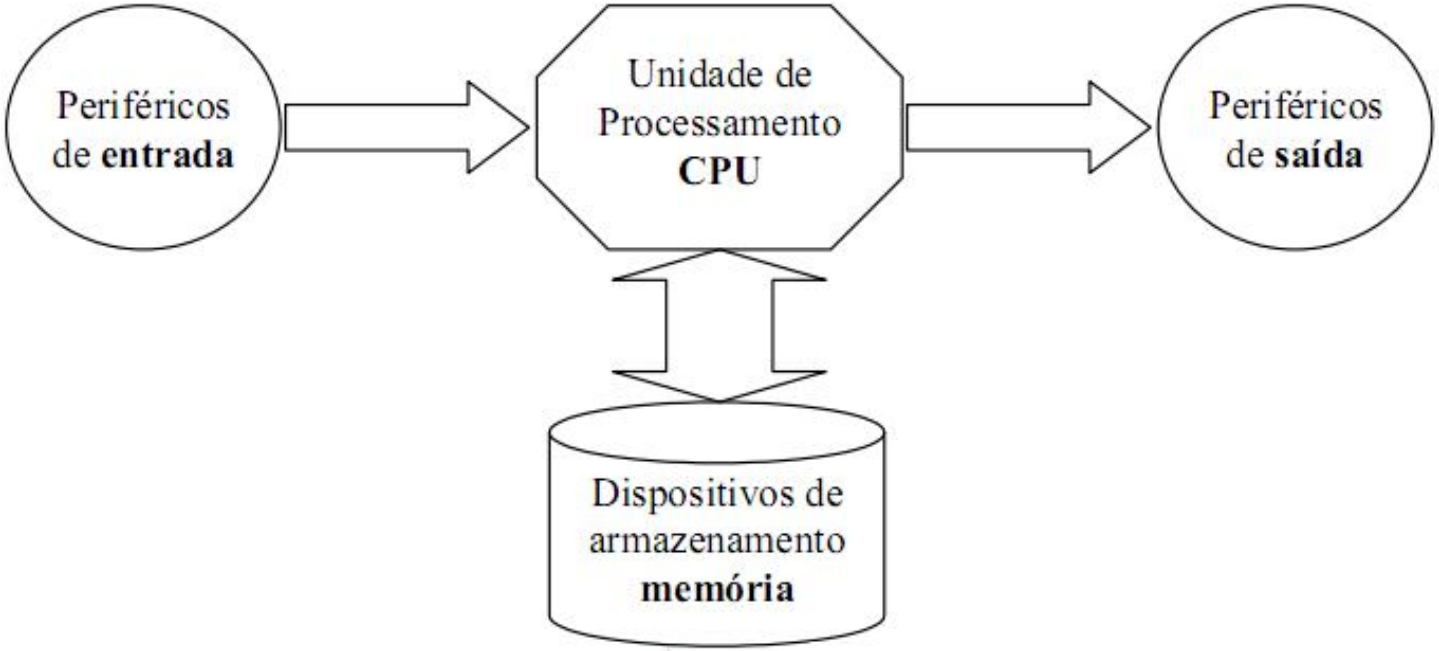


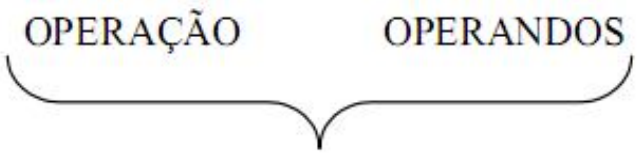
# Organização Básica

- **CPU (Unidade Central de Processamento - processador)**
  - **ULA (Unidade Lógica e Aritmética)**
  - **UC (Unidade de Controle)**
  - **Registradores:**
    - **Apontador de programa (PC) ,**
    - **Apontador de Pilha (SP) ,**
    - **Acumulador, Uso geral, Status, ..)**
- **Memória:**
  - **RAM, ROM, PROM, EPROM,**
- **E/S :**
  - **Interfaces para comunicação com dispositivos externos (série e paralelo)**
- **Barramento : Dados (D0 - Dn ) , endereço (A0 - An ) e controle.->.**

**Interliga mais do que dois dispositivos, senão trata-se apenas de uma ligação ponto a ponto**

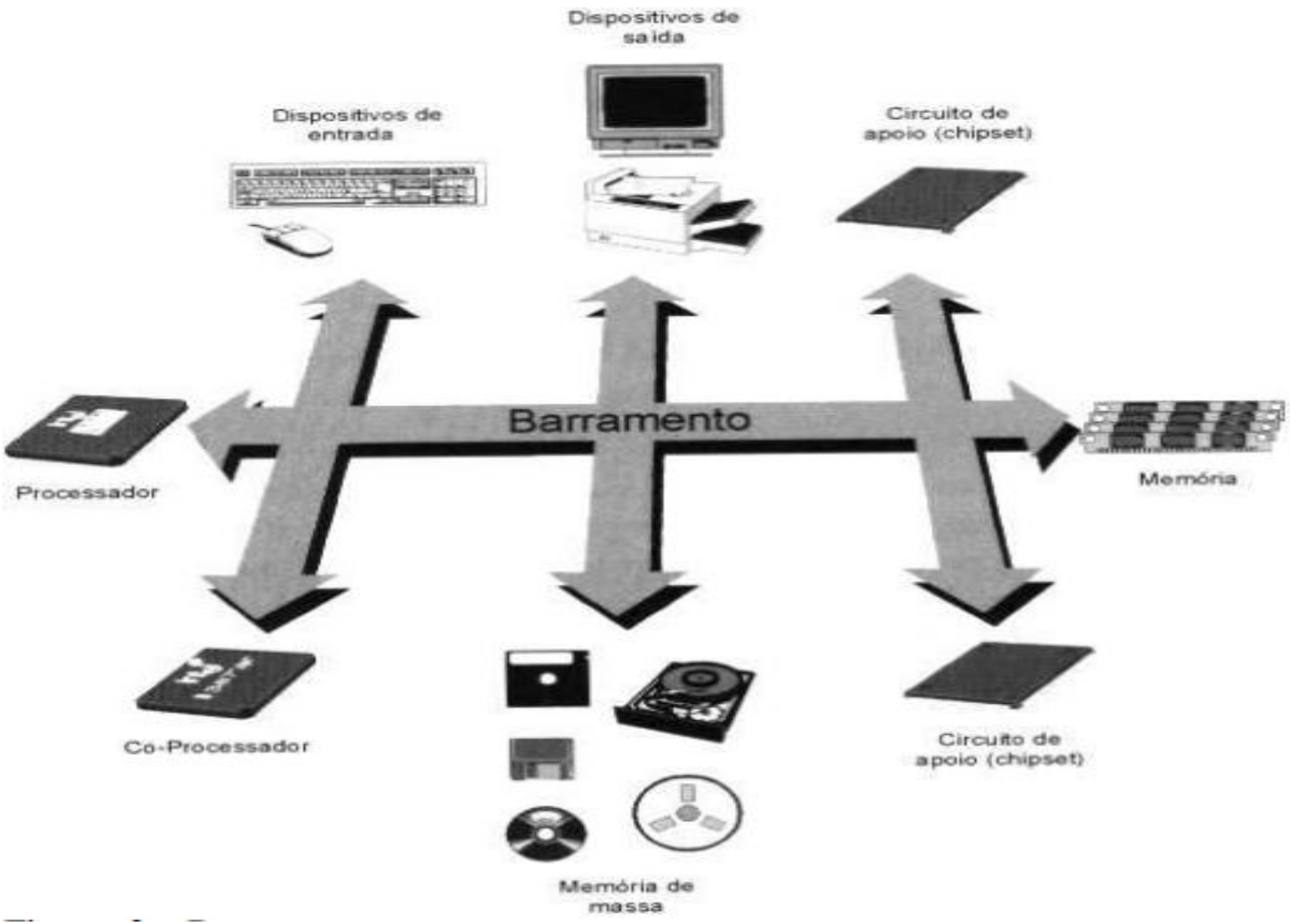
# Organização Básica



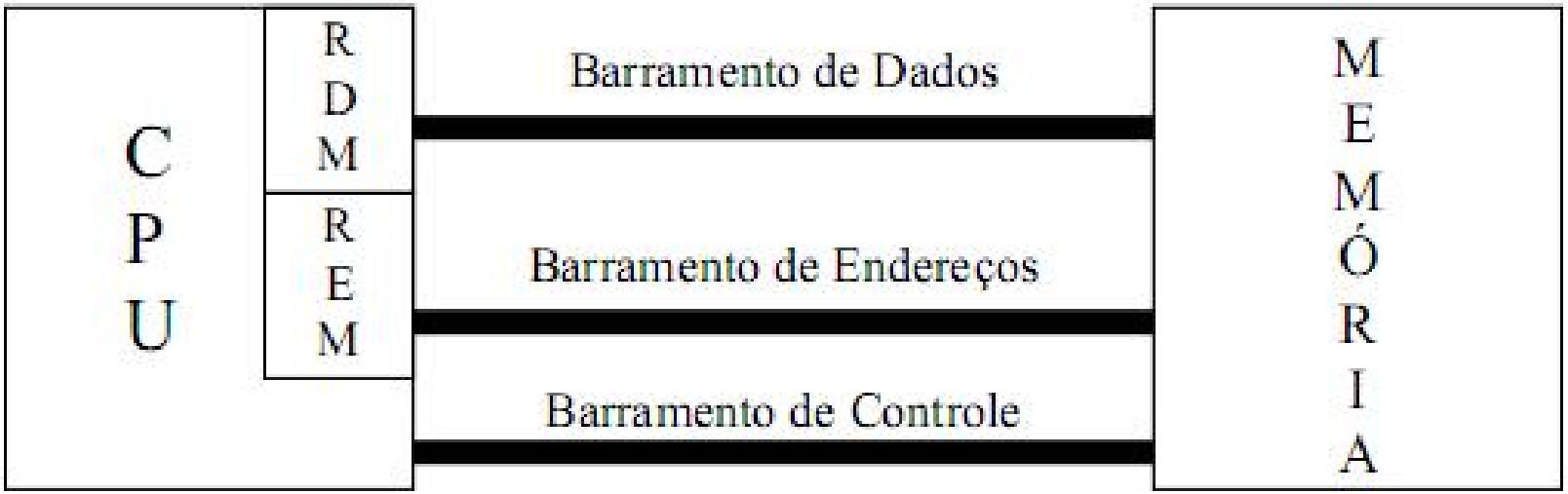




**INSTRUÇÃO**

43


Dispositivos de saída
Dispositivos de entrada
Circuito de apoio (chipset)
Barramento
Processador
Memória
Co-Processador
Circuito de apoio (chipset)
Memória de massa

# Organização Básica - barramento

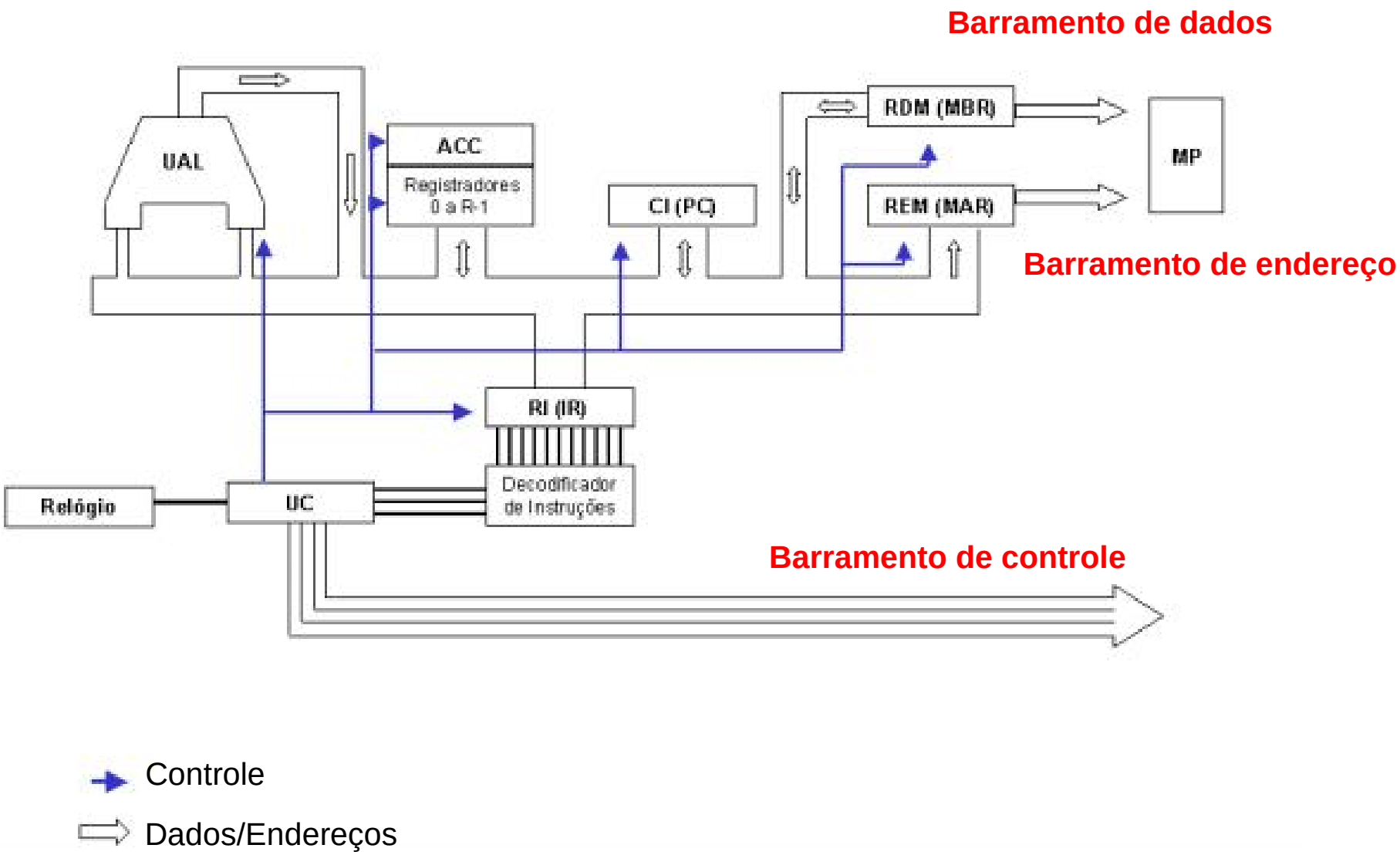



- Mostrar exemplos no quadro com RD e WR

# Operações Básicas - CPU

- **Operações realizadas pela CPU**
  - **Funções de Processamento**
    - operações aritméticas e lógicas
    - movimentação de dados
    - desvios
    - operações de entrada ou saída
  - **Funções de Controle**
    - Busca, interpretação e controle da execução das instruções.
    - Controle da ação dos demais componentes do sistema de computação (memória, entrada/saída).

# Exemplo de CPU



# Esquema básico de um computador

- A Unidade de Controle e a Unidade Lógica e Aritmética constituem a Unidade Central de Processamento.

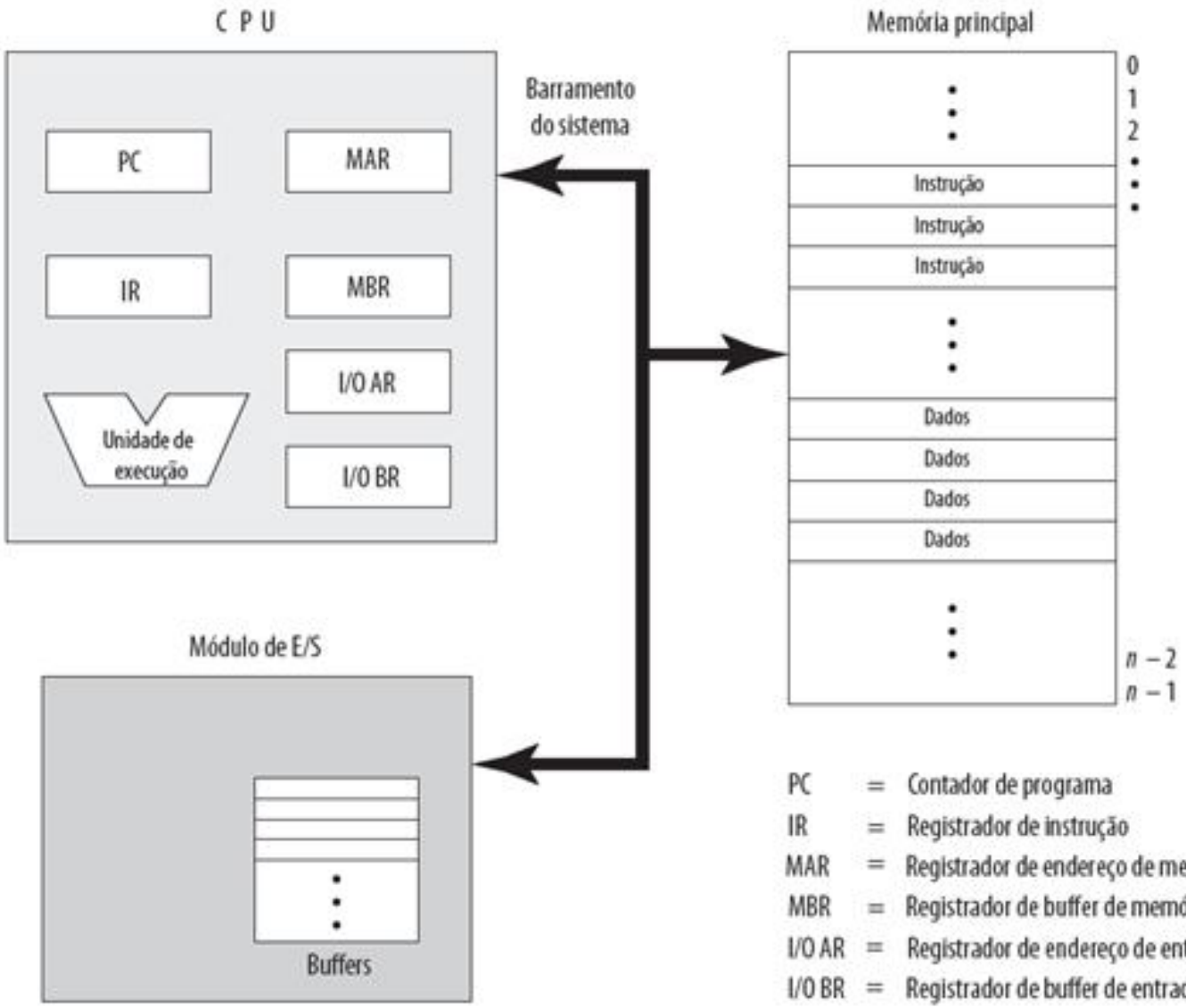
- Dados e instruções precisam entrar no sistema, e resultados saem dele.
  - Entrada/saída.

- É necessário um armazenamento temporário de código e resultados.
  - Memória principal.

# Componentes fundamentais da CPU

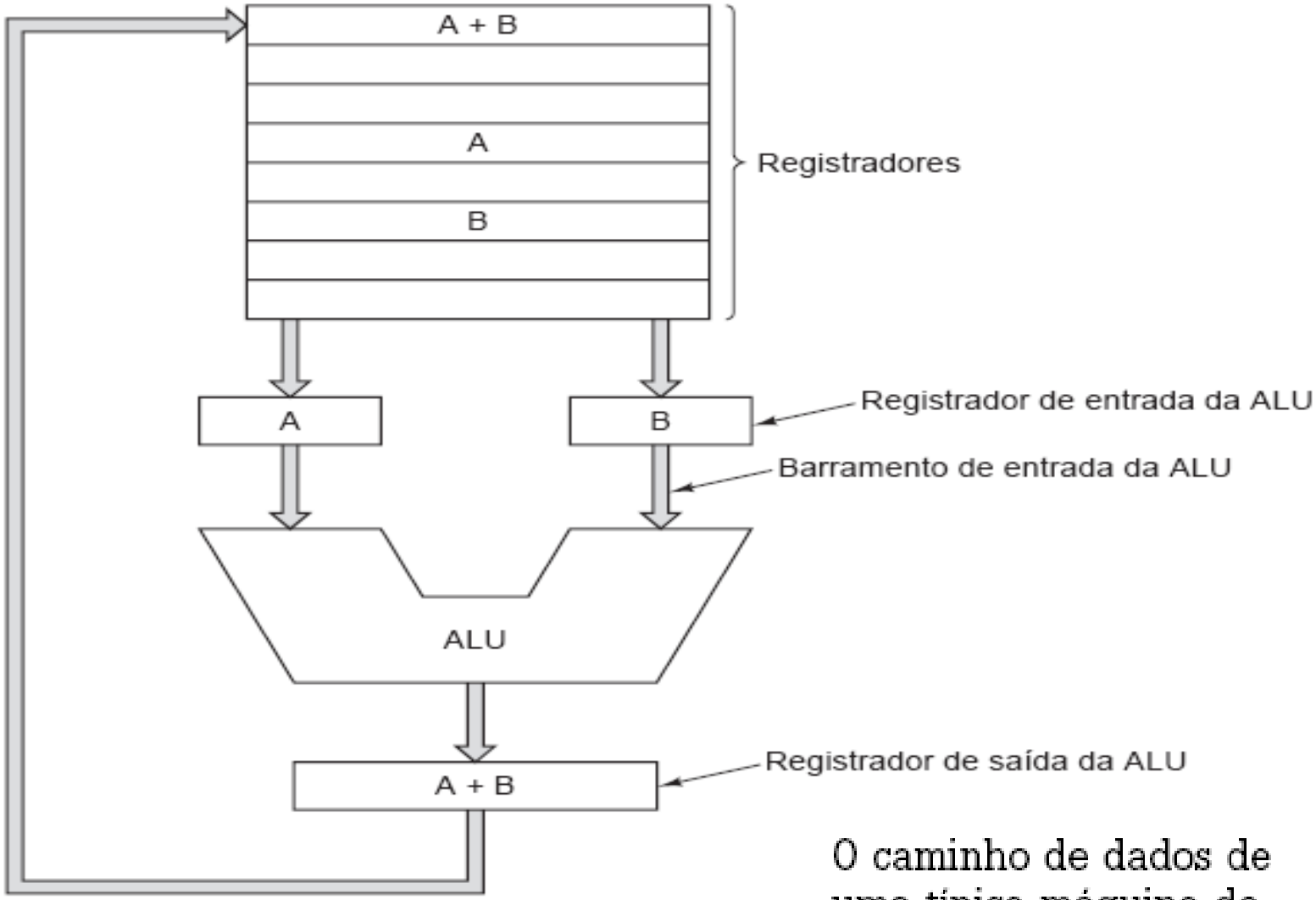
- Unidade de Controle (UC)
- Unidade Lógica e Aritmética (ULA)
- Registradores auxiliares
- Contador de programa (PC)
- Registrador de dados (RDM)
- Registrador de endereço (REM)
- Registrador de Instrução (RI)
- Controlador do barramento
- Decodificador de instrução

# Visão de alto nível (componentes do computador)




PC = Contador de programa
IR = Registrador de instrução
MAR = Registrador de endereço de memória
MBR = Registrador de buffer de memória
I/O AR = Registrador de endereço de entrada/saída
I/O BR = Registrador de buffer de entrada/saída

# Organização da CPU

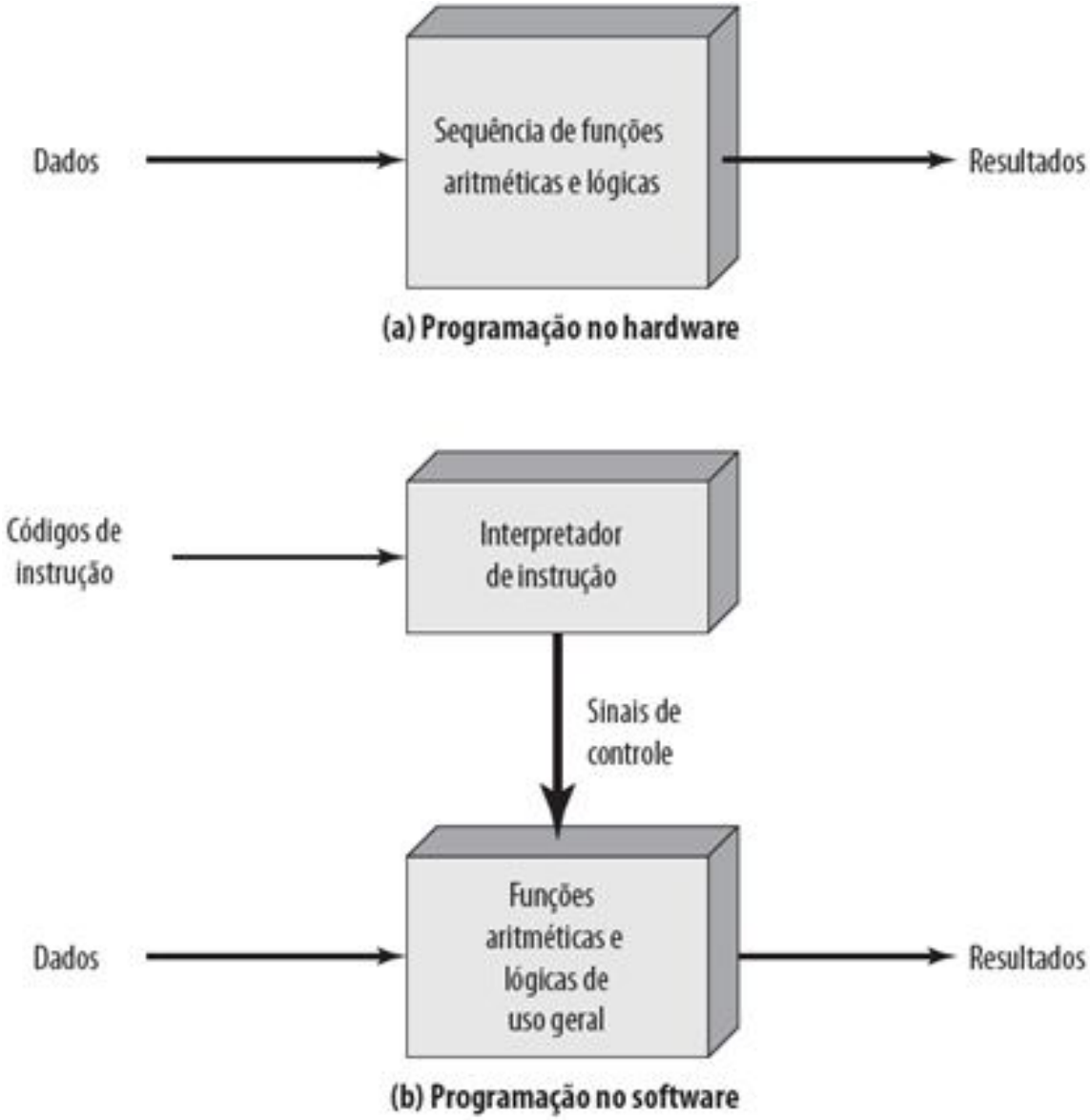



O caminho de dados de uma típica máquina de Von Neumann.

# Abordagem de hardware e software

- Sistemas “hardware” são inflexíveis.
- Hardware de uso geral pode fazer diferentes tarefas, dados sinais de controle corretos.
- Ao invés de religar o hardware, forneça um conjunto de sinais de controle.

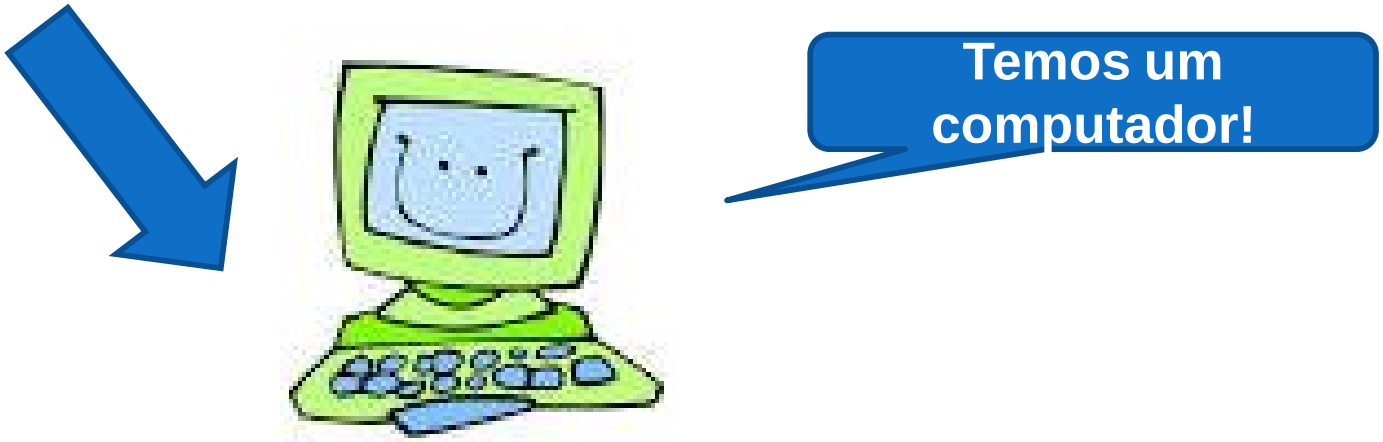
# Abordagem de hardware e software



# Função da unidade de controle

- Para cada operação, um código exclusivo é fornecido.
  - Ex. **ADD** tem um código binário. **MOVE** tem outro código.

- Um segmento de hardware aceita o código e emite os sinais de controle.

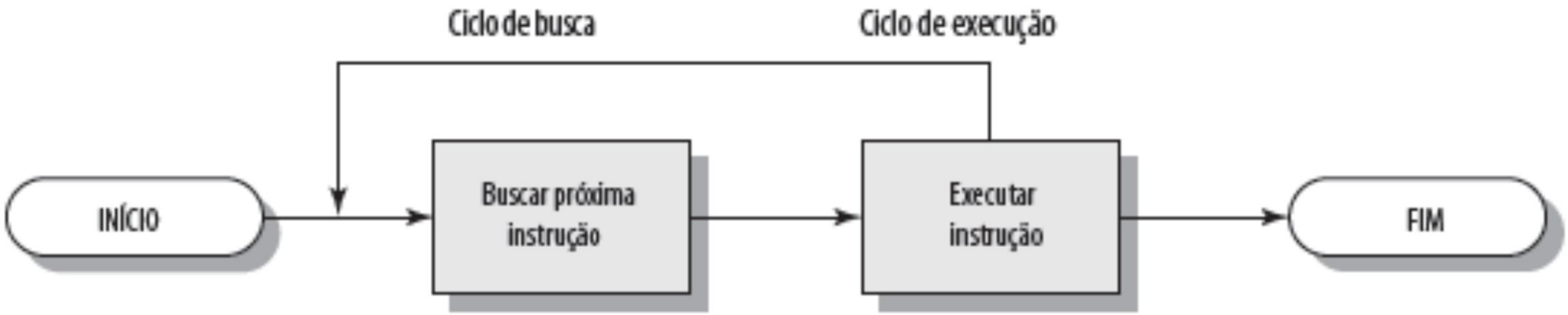


# O que é um programa?

- Uma sequência de etapas.
- Para cada etapa, é feita uma operação aritmética ou lógica.
- Para cada operação, é necessário um conjunto diferente de sinais de controle.

# Ciclo de instrução

- Duas etapas:
  - Busca
  - Execução



# Ciclo de busca

- Contador de Programa (PC) mantém endereço da próxima instrução a buscar.
- Processador busca instrução do local de memória apontado pelo PC.
- Incrementar PC:
  - A menos que seja informado de outra forma.
- Instrução carregada no Registrador de Instrução (IR).
- Processador interpreta instrução e realiza ações exigidas.

# Ciclo de execução

- Processador-memória:
  - Transferência de dados entre CPU e memória principal.
- E/S do processador:
  - Transferência de dados entre CPU e módulo de E/S.
- Processamento de dados:
  - Alguma operação aritmética ou lógica sobre dados.
- Controle:
  - Alteração da sequência de operações.
  - P.e. salto.
- Combinação dos itens anteriores.

# Execução total de uma instrução

- Trazer a próxima instrução da memória até o registrador.
- Alterar o contador de programa para indicar a próxima instrução.
- Identificar o tipo de instrução trazida
- Se a instrução usar uma palavra de memória, determinar onde essa palavra está
- Trazer a palavra para dentro de um registrador da CPU, se necessário
- Executar a instrução
- Voltar a etapa 1 para iniciar a execução da instrução seguinte

# Exemplo hipotético do formato de uma instrução

**(a) Formato de instrução**

**(b) Formato de inteiro**

Contador de programa (PC) = Endereço da instrução
Registrador de instrução (IR) = Instrução sendo executada
Acumulador (AC) = Armazenamento temporário

**(c) Registradores internos da CPU**

0001 = Carrega AC da memória
0010 = Armazena AC na memória
0101 = Adiciona da memória ao AC

**(d) Lista parcial de *opcodes***

# Formato de instruções (Campos)

| Código de operação (opcode) | (identificação do operando) |
|---|---|

No exemplo anterior:

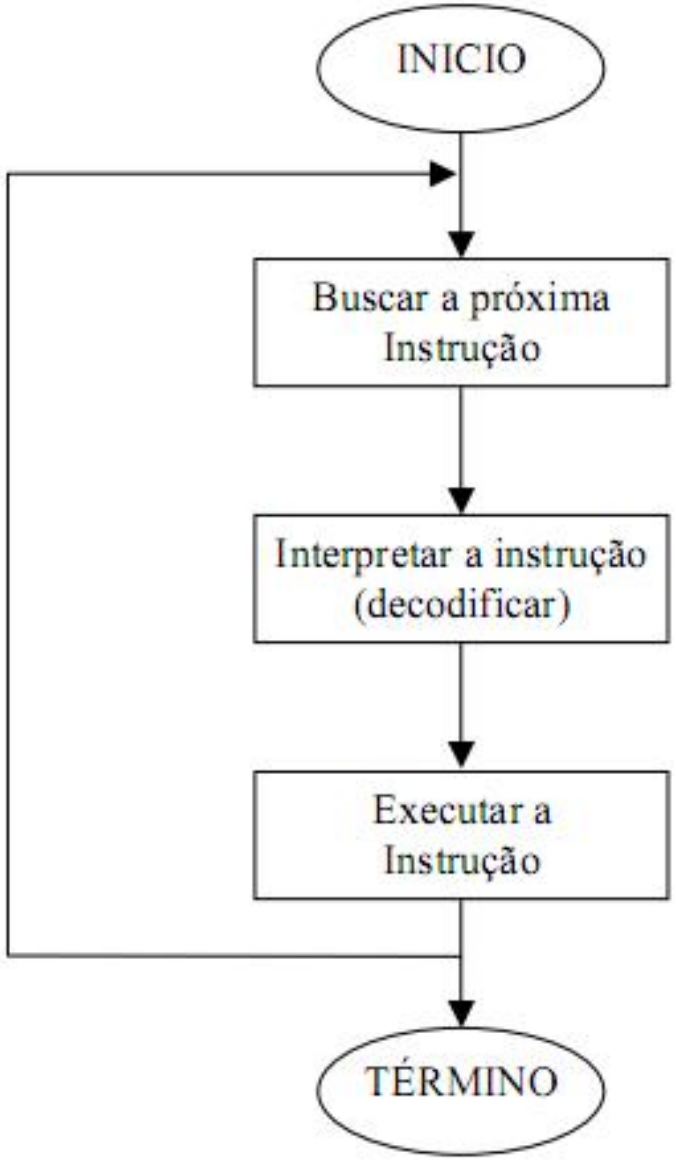
- 0001 -> carregar AC a partir do endereço de memória especificado .
- 0010 -> armazena o valor contido em AC no endereço de memória especificado
- 0101 -> acrescenta ao valor contido em AC o valor contido no endereço de memória especificado

Por exemplo:

- Operando imediato
- Endereço do operando

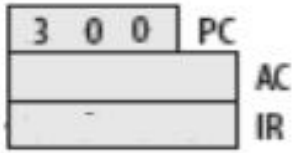
- Instruções de E/S específicas, exige barramento e *opcode* de E/S, neste caso 0 dispositivo não é mapeado na memória

# Ciclo de Instrução



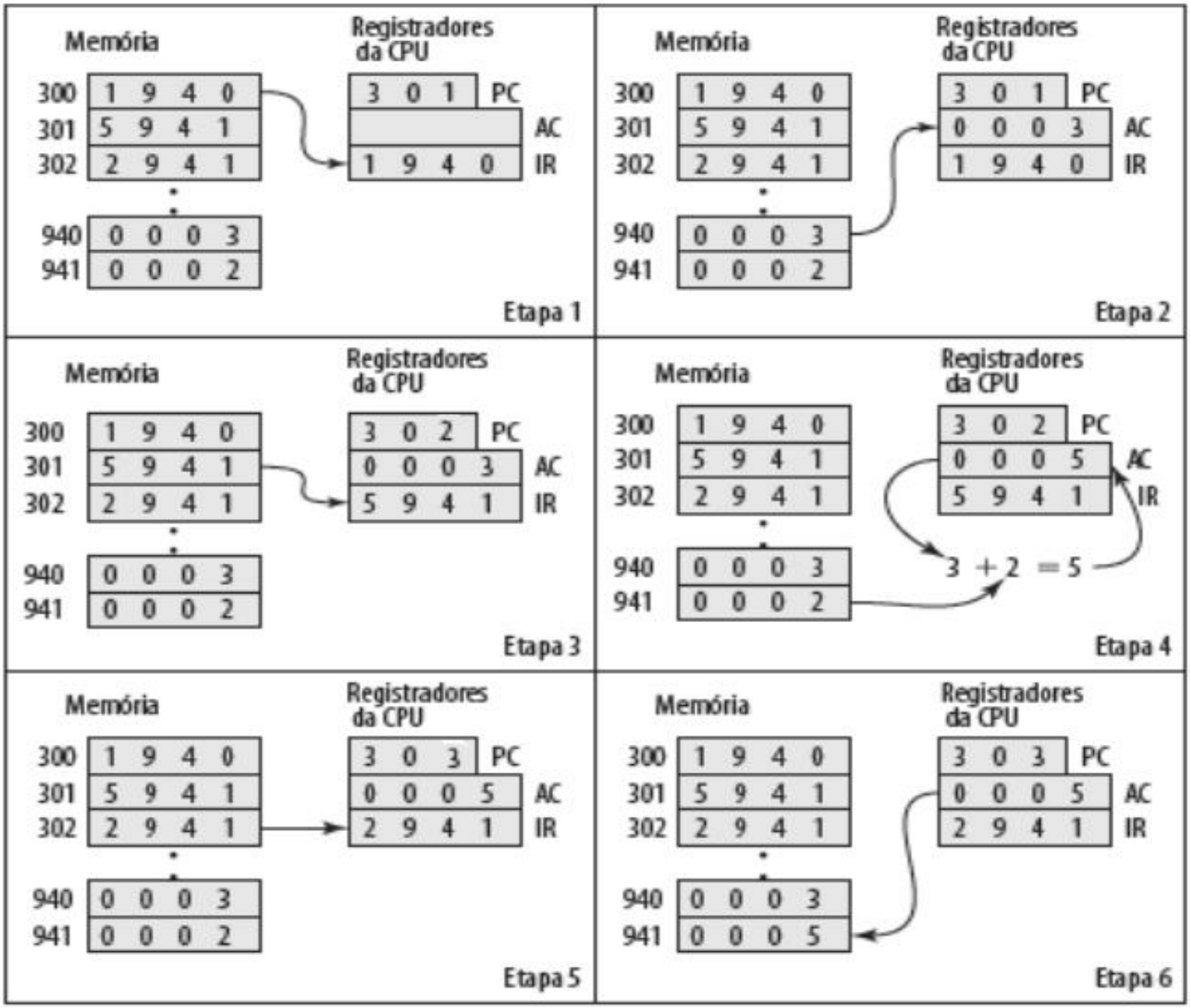
# Exemplo de execução de programa

Inicio

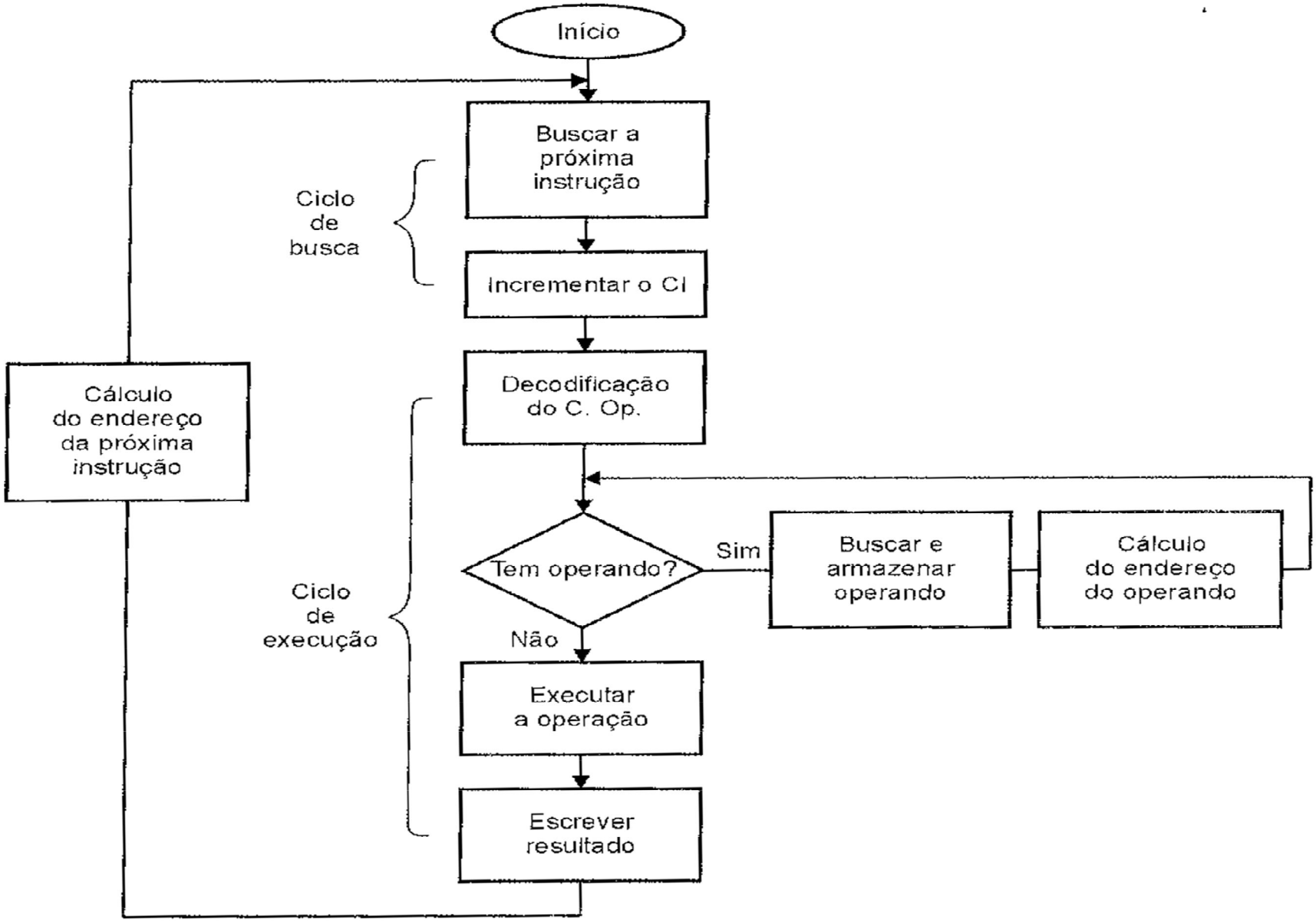



Busca

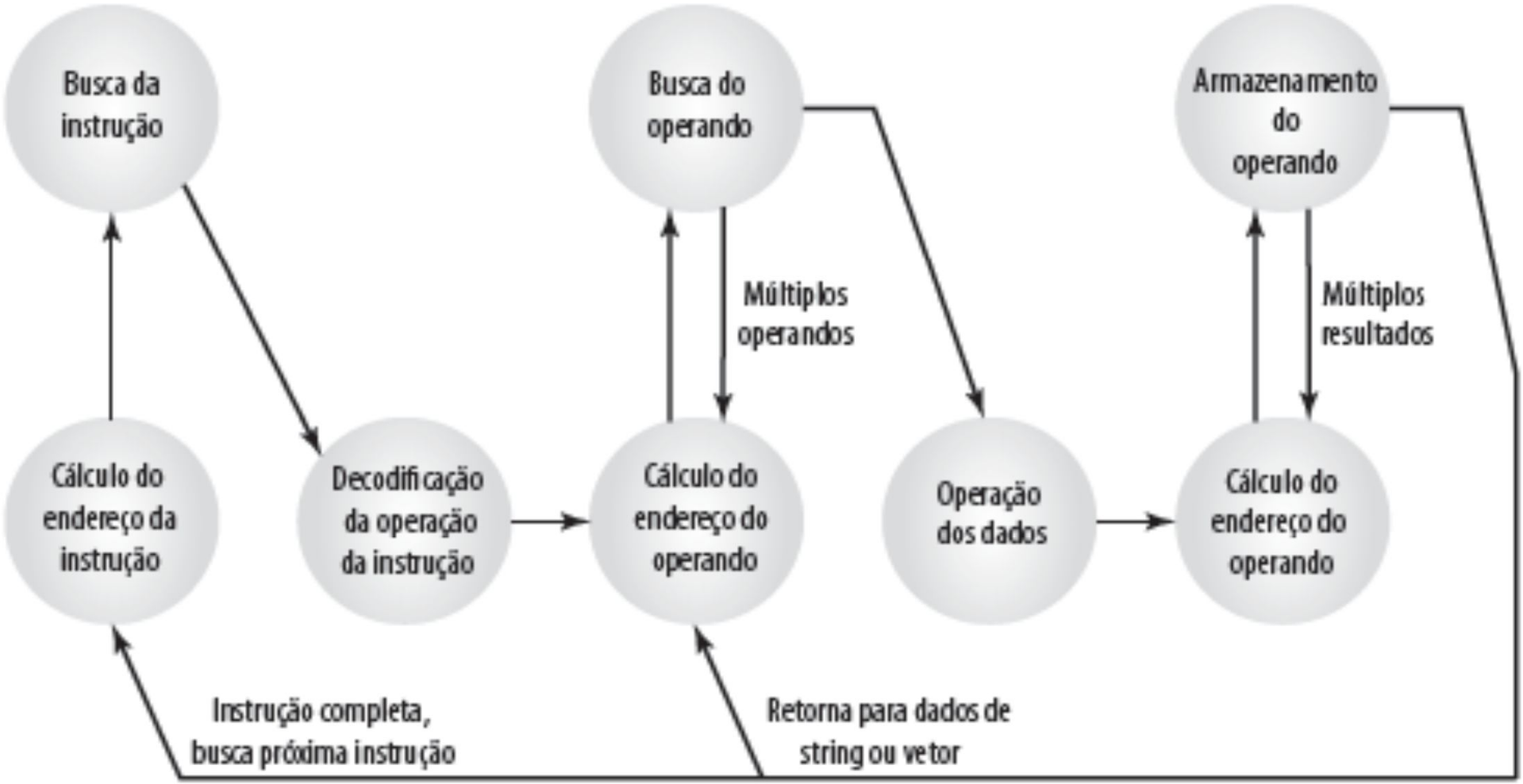
Execução



# Ciclo de Instrução - detalhando



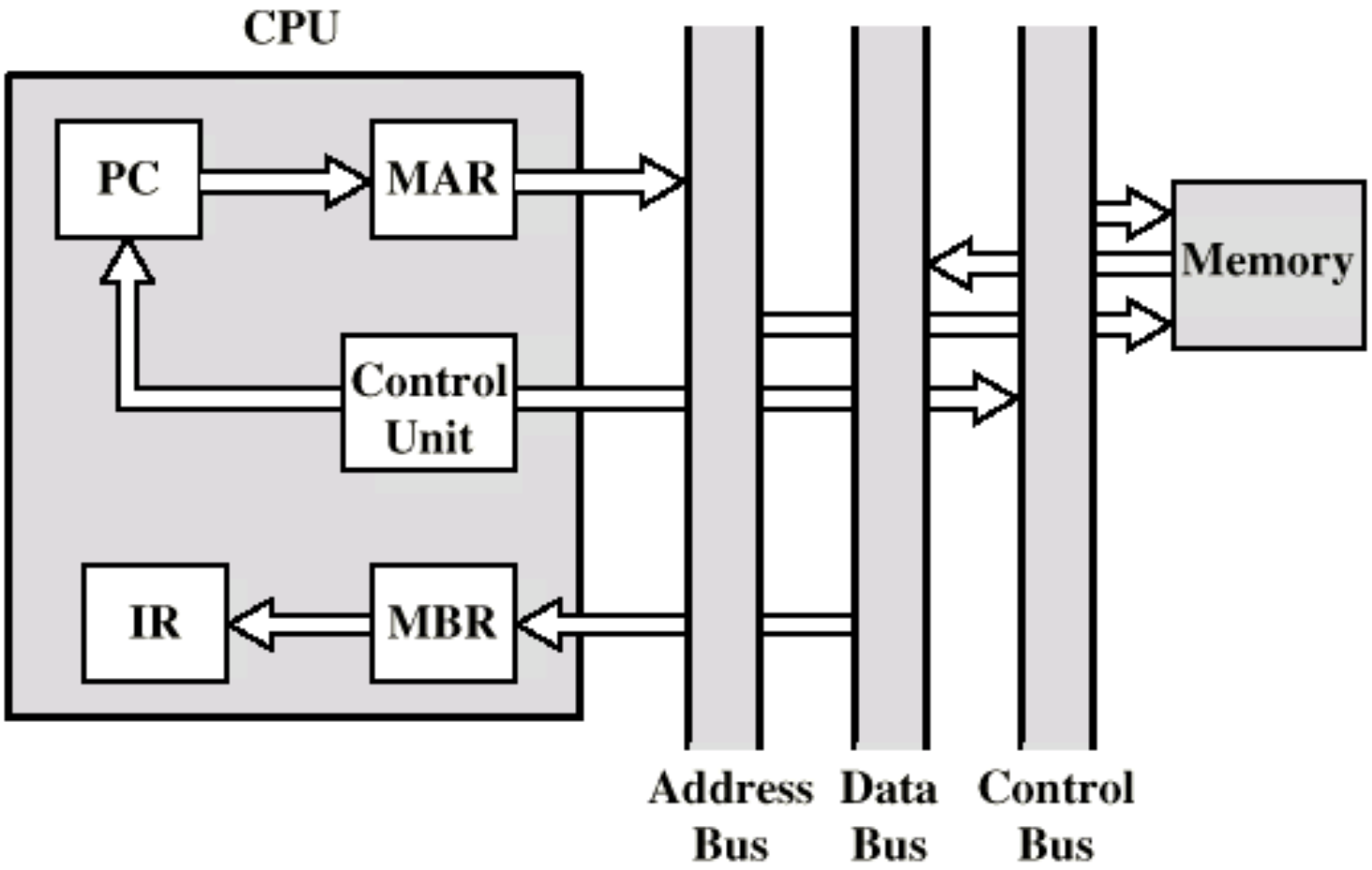
# Diagrama de estado do ciclo de instrução



# Revisando: Estado do ciclo de instrução

- **Busca da Instrução**: lê a instrução na memória
- **Cálculo de endereço**: determina o endereço da próxima instrução
- **Decodificação da operação**: identifica a instrução (operação)
- **Cálculo do endereço do operando**: localiza o operando
- **Busca do Operando**: busca o operando
- **Operação dos dados**: realiza a operação indicada na instrução
- **Armazenamento do operando**: salvar o resultado

# Fluxo de dados no fetch (busca)




MBR = Memory buffer register
MAR = Memory address register
IR = Instruction register
PC = Program counter

# Registradores usados no fetch (busca)

- **Registrador de endereço de memória (MAR):** conectado às linhas de endereço do barramento
- **Registrador de buffer de memória (MBR):** conectado às linhas de dados do barramento
- **Contador de programa (PC):** guarda o endereço da próxima instrução
- **Registrador de instrução (IR):** guarda a instrução